A project report on

“MULTI-FUNCTION ROBOT CAR USING ARDUINO UNO”

Project report submitted
In partial fulfilment for the requirement for the award of
DIPLOMA IN ELECTRICAL ENGINEERING

Submitted By –

Dus Mamud	CHP/23/EL/020
Iswar Ch. Das	CHP/23/EL/025
Bhardwaj Sarkar	CHP/23/EL/008
Anup Das	CHP/23/EL/005
Dibyajyoti Das	CHP/23/EL/016
Nazrul Islam	CHP/23/EL/046
Sourab Sarkar	CHP/23/EL/059

Under the guidance of –
Ms. Chinmayee Medhi
Lecturer, Electrical Engineering

DEPARTMENT OF ELECTRICAL ENGINEERING, CHIRANG POLYTECHNIC, BIJNI, ASSAM – 783390

26th May, 2026

CERTIFICATE OF APPROVAL (HEAD OF DEPARTMENT)

The Final project report on "**MULTI-FUNCTION ROBOT CAR USING ARDUINO UNO**" has been prepared and submitted by:

Dus Mamud (CHP/23/EL/020), Iswar Ch. Das (CHP/23/EL/025), Bhardwaj Sarkar (CHP/23/EL/008), Anup Das (CHP/23/EL/005), Dibyajyoti Das (CHP/23/016), Nazrul Islam (CHP/23/EL/046) and **Sourab Sarkar CHP/23/EL/059)**. They are students of 6th Semester of Electrical Engineering Department of Chirang Polytechnic, Bijni Assam and this project report has carried as a partial fulfilment of requirement of Diploma in Electrical Engineering.

Signature of H.O.D

Mr. Amit Kumar Das

Date :

Place :

CERTIFICATE BY SUPERVISOR

The Final project report on "**MULTI-FUNCTION ROBOT CAR USING ARDUINO UNO**" has been prepared and submitted by:

Dus Mamud (CHP/23/EL/020), Iswar Ch. Das (CHP/23/EL/025), Bhardwaj Sarkar (CHP/23/EL/008), Anup Das (CHP/23/EL/005), Dibyajyoti Das (CHP/23/016), Nazrul Islam (CHP/23/EL/046) and **Sourab Sarkar CHP/23/EL/059)**. They are students of 5th Semester from the Department of Electrical Engineering, Chirang Polytechnic, Bijni Assam and they had completed their work with dedication and hard work. I wish best of luck for their bright future.

Signature of Supervisor **Signature of H.O.D**

Ms. Chinmayee Medhi Mr. Amit Kumar Das

Date :

Place :

CERTIFICATE OF DECLARATION

We, **Dus Mamud (CHP/23/EL/020), Iswar Ch. Das (CHP/23/EL/025), Bhardwaj Sarkar (CHP/23/EL/008), Anup Das (CHP/23/EL/005), Dibyajyoti Das (CHP/23/016), Nazrul Islam** and **(CHP/23/EL/046) Sourab Sarkar CHP/23/EL/059)** declare That the work presented in this professional project report entitled "**MULTI-FUNCTION ROBOT CAR USING ARDUINO UNO**" submitted in the Electrical Engineering Department of Chirang Polytechnic, Bijni Assam is an authentic record of our own work and has not been submitted by us for any other Degree or Diploma.

Date :

Place :

Submitted By: **Signature:**

Name:	**Roll No.**
Dus Mamud	(CHP/23/EL/020)
Iswar Ch. Das	(CHP/23/EL/025)
Bhardwaj Sarkar	(CHP/23/EL/008)
Anup Das	(CHP/23/EL/005)
Dibyajyoti Das	(CHP/23/EL/016)
Nazrul Islam	(CHP/23/EL/046)
Sourab Sarkar	(CHP/23/EL/059)

TABLE OF CONTENTS

ACKNOWLEDGEMENT

At the very beginning, we would like to express our sincere thanks and gratitude to our guide **Ms. Chinmayee Medhi**, Lecturer in Electrical Engineering Department for his encouragement during this project. We are also thankful to our Principal **JITU MONI DEKA** and all Faculties of Electrical Engineering Department. We are also thankful to other Faculties of Chirang Polytechnic.

Date :

Place :

Submitted By: **Signature:**

Name:	**Roll No.**
Dus Mamud	(CHP/23/EL/020)
Iswar Ch. Das	(CHP/23/EL/025)
Bhardwaj Sarkar	(CHP/23/EL/008)
Anup Das	(CHP/23/EL/005)
Dibyajyoti Das	(CHP/23/EL/016)
Nazrul Islam	(CHP/23/EL/046)
Sourab Sarkar	(CHP/23/EL/059)

ABSTRACT

The rapid advancement of embedded systems and wireless communication technologies has enabled the development of intelligent, remotely controlled robotic systems. This project, titled "**Multi-Function Robot Car Using Arduino UNO**," aims to design and implement a cost-effective, multi-mode robot car capable of operating in three distinct functional modes – Obstacle Avoidance, Bluetooth Manual Control, and Voice Control – all integrated within a single Arduino-based platform..

The system is built using an **Arduino UNO microcontroller** as the central processing unit, combined with an **L293D Motor Driver Shield** to drive four DC gear motors, an **HC-SR04 Ultrasonic Sensor** for real-time obstacle detection, an **SG90 Servo Motor** for directional scanning, and an **HC-05 Bluetooth Module** for wireless communication with an Android smartphone.

In **Obstacle Avoidance Mode**, the robot autonomously navigates its environment by scanning left and right using the servo-mounted ultrasonic sensor and selecting the direction with greater clearance when an obstacle is detected within 12 cm. In **Bluetooth Manual Control Mode**, the user controls the robot's movement (Forward, Backward, Left, Right, Stop) through a smartphone application that transmits single-character commands (F, B, L, R, S) over Bluetooth. In **Voice Control Mode**, spoken commands such as "Forward," "Backward," "Left," "Right," and "Stop" are recognized by the mobile application and converted into corresponding serial characters (^, -, <, >, *) sent to the Arduino, with an added safety feature that checks the clearance distance before executing Left or Right turns.

The robot chassis is constructed using foam board and cardboard, powered by two Li-ion 3.7V batteries, making the overall system compact, lightweight, and affordable. The complete Arduino program is structured with three separate modular functions – Obstacle(), Bluetoothcontrol(), and voicecontrol() – which can be individually activated by commenting or uncommenting within the main loop() function.

The result is a fully functional, versatile, and interactive robot platform that demonstrates practical applications of wireless communication, autonomous

navigation, and human-machine interaction using low-cost, readily available components. This project has potential applications in educational robotics, home automation assistance, basic surveillance, and serves as a foundational prototype for more advanced autonomous systems.

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATION

S. No.	Abbreviation	Full Form
1	AGV	Automated Guided Vehicle
2	ADAS	Advanced Driver Assistance System
3	API	Application Programming Interface
4	AEB	Automatic Emergency Braking
5	AVR	Alf and Vegard's RISC Processor
6	BT	Bluetooth
7	DC	Direct Current
8	DIY	Do It Yourself
9	EDR	Enhanced Data Rate
10	EEPROM	Electrically Erasable Programmable Read-Only Memory
11	GND	Ground
12	GPIO	General Purpose Input / Output
13	GUI	Graphical User Interface
14	HC-05	Hc (Host Controller) – Serial Bluetooth Module (Model 05)
15	HC-SR04	Hc (Model) – SR04 Ultrasonic Distance Sensor
16	HMI	Human-Machine Interface
17	I/O	Input / Output
18	IC	Integrated Circuit
19	IDE	Integrated Development Environment
20	IoT	Internet of Things
21	IR	Infrared

22	ISM	Industrial, Scientific, and Medical (frequency band)
23	LIDAR	Light Detection and Ranging
24	PWM	Pulse Width Modulation
25	RAM	Random Access Memory
26	RF	Radio Frequency
27	RISC	Reduced Instruction Set Computer
28	RPM	Revolutions Per Minute
29	SLAM	Simultaneous Localisation and Mapping
30	SPI	Serial Peripheral Interface
31	SPP	Serial Port Protocol
32	SRAM	Static Random Access Memory
33	STEM	Science, Technology, Engineering and Mathematics
34	ToF	Time of Flight
35	TRIG	Trigger (Pin on Ultrasonic Sensor)
36	ECHO	Echo (Pin on Ultrasonic Sensor)
37	UART	Universal Asynchronous Receiver-Transmitter
38	USB	Universal Serial Bus
39	VCC	Voltage Common Collector (Power Supply Pin)
40	Wi-Fi	Wireless Fidelity

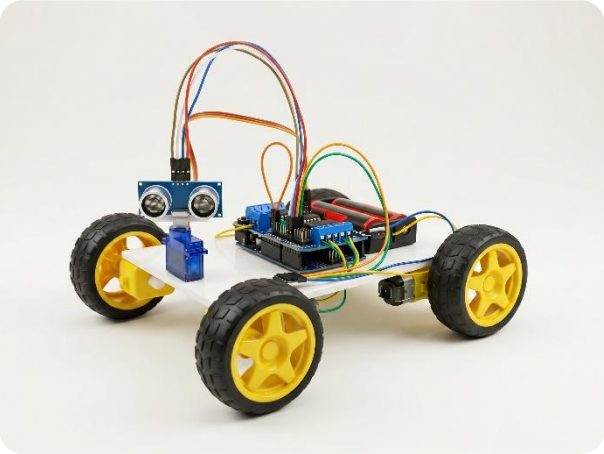
LIST OF SYMBOLS

S. No.	Symbol	Meaning	Unit
1	V	Voltage	Volts (V)
2	I	Current	Amperes (A)
3	mA	Milliampere	10^{-3} Amperes (A)
4	mAh	Milliampere-Hour	Unit of Battery Capacity
5	R	Resistance	Ohms (Ω)
6	W	Power	Watts (W)
7	Hz	Frequency	Hertz (Hz)
8	kHz	Kilohertz	10^{3} Hertz (Hz)
9	MHz	Megahertz	10^{6} Hertz (Hz)
10	GHz	Gigahertz	10^{9} Hertz (Hz)
11	t	Time / Pulse Duration	Seconds (s) / Microseconds (µs)
12	µs	Microsecond	10^{-6} Seconds (s)
13	ms	Millisecond	10^{-3} Seconds (s)
14	cm	Centimetre	Unit of Distance
15	m	Metre	Unit of Distance
16	m/s	Metres per Second	Unit of Speed / Velocity
17	cm/µs	Centimetres per Microsecond	Unit of Ultrasonic Wave Speed
18	°	Degree	Unit of Angle (Servo Position)
19	kg·cm	Kilogram-Centimetre	Unit of Torque
20	g	Gram	Unit of Mass (Component Weight)
21	KB	Kilobyte	10^{3} Bytes – Unit of Memory
22	%	Percentage	Ratio expressed per hundred

23	≤	Less than or equal to	Mathematical Comparison Operator
24	≥	Greater than or equal to	Mathematical Comparison Operator
25	×	Multiplication	Mathematical Operator
26	÷	Division	Mathematical Operator

Chapter 1 Introduction

1.1 Overview of Multi-Function Robots



In the modern era of technology and automation, robotics has emerged as one of the most dynamic and rapidly evolving fields of engineering. A **robot** is an automatically operated machine that replaces human effort, though it may not particularly resemble human beings in appearance or perform functions in a human-like manner. Over the past few decades, robots have found their applications in a wide range of domains – from industrial manufacturing and medical surgery to space exploration and household assistance.

Amongst the various categories of robots, **wheeled mobile robots** have gained significant popularity in the field of embedded systems and educational electronics. These robots are relatively simple to design, cost-effective to build, and highly versatile in their applications. A **multi-function robot** car is a wheeled mobile robot that can operate in more than one mode of control, making it adaptable to different scenarios and user requirements.

This project, titled "**MULTI-FUNCTION ROBOT CAR USING ARDUINO UNO**," presents the design, construction, and implementation of a three-mode intelligent robot car. The three operating modes are:

- **Obstacle Avoidance Mode** – The robot navigates autonomously by detecting obstacles using an ultrasonic sensor and choosing a clear path.
- **Bluetooth Manual Control Mode** – The user manually drives the robot using a smartphone application over a Bluetooth wireless link.
- **Voice Control Mode** – The user gives spoken commands such as *"Forward," "Backward," "Left," "Right,"* and *"Stop,"* which are recognised by the smartphone and transmitted to the robot.

The entire system is built upon the **Arduino UNO microcontroller** – an open-source, beginner-friendly yet powerful platform based on the ATmega328P chip. The **L293D Motor Driver Shield** is used to interface and drive the four DC gear motors that propel the robot. An **HC-SR04 Ultrasonic Sensor** mounted on an **SG90 Servo Motor** provides the robot with its "eyes" – the ability to measure distances and scan the surrounding environment. The **HC-05 Bluetooth Module** acts as the wireless bridge between the Android smartphone and the robot.

This project is a natural and direct advancement of our previous semester's work – "**RADAR SYSTEM USING ARDUINO UNO**" – in which we utilised the same HC-SR04 ultrasonic sensor and SG90 servo motor for environmental mapping.

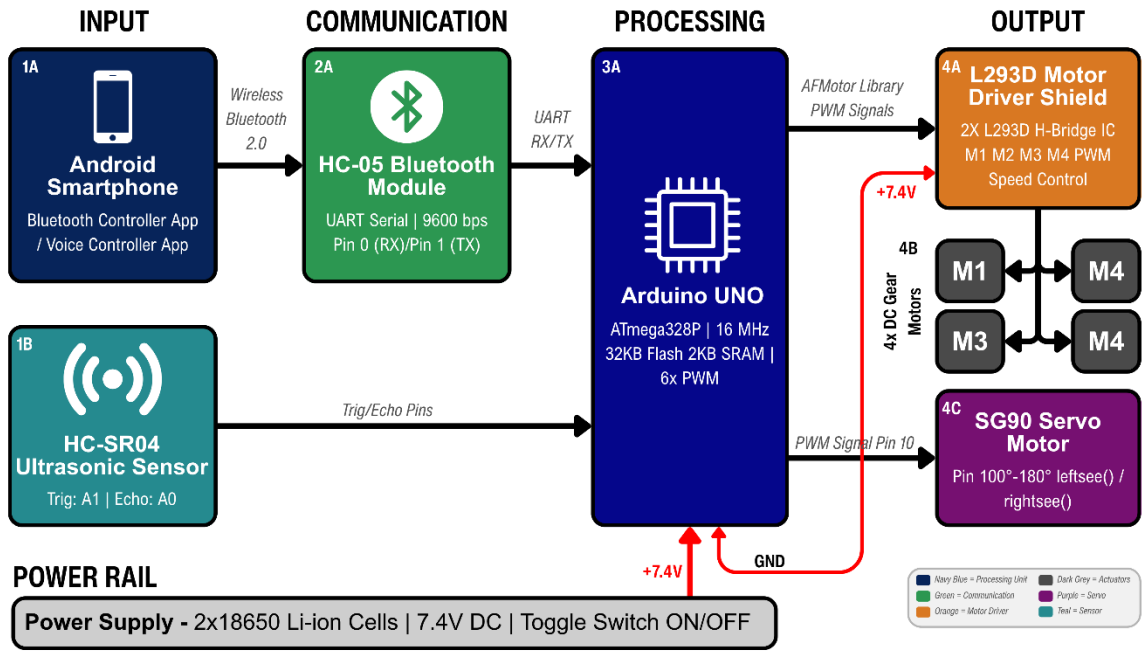


Figure 1.1 – Block Diagram of the Multi-Function Robot System

In this current project, we have extended those concepts further by incorporating mobility, wireless communication, and human-voice interaction into a complete robotic platform.

1.2 Problem Statement

Despite the remarkable progress in robotics and automation, several practical challenges persist – particularly in the context of affordable, accessible, and multi-modal control systems for small-scale robots. The following specific problems motivated the development of this project:

1. **Single-Mode Limitation in Existing Low-Cost Robots**: Most low-cost, Arduino-based robot kits available in the educational market are designed to perform only a single function – either obstacle avoidance or Bluetooth control or voice control. They lack the flexibility to switch between modes based on the user's need or the environment's demand. A robot that can only avoid obstacles cannot be manually guided to a specific target, and a robot that only responds to Bluetooth commands cannot protect itself from collisions when the user is inattentive.

2. **Lack of Safety in Wireless-Controlled Robots**: Conventional Bluetooth-controlled toy cars or robot kits execute movement commands blindly – they move in whatever direction the user commands, even if an obstacle is directly in the path. This results in frequent collisions, damage to the robot, and potential injury. There is a need for a robot that incorporates intelligent safety checks, especially for directional voice commands.

.

3. **Absence of Voice-Based Human-Robot Interaction**: Most affordable robot platforms in the educational domain rely entirely on button-based control via smartphone apps. Voice-based

interaction, despite being a natural and intuitive mode of communication, is rarely implemented in low-cost student-level robotics projects. This creates a gap between the theoretical knowledge of serial communication and its practical, human-centric application.

4. **High Cost of Advanced Robotic Platforms**: Professional multi-mode robots that offer autonomous navigation, wireless control, and voice interaction simultaneously are prohibitively expensive and involve complex hardware architectures that are beyond the scope of a Diploma-level project. There is a clear requirement for a low-cost prototype that demonstrates all three functionalities using simple, readily available components.

Therefore, this project addresses all four of the above problems by constructing a single robot platform that supports autonomous obstacle avoidance, Bluetooth manual control, and safety-aware voice control – all implemented using affordable, commonly available electronic components.

1.3 Objectives of the Project

The primary aim of this project is to design, build, and program a multi-function robot car that integrates three distinct control systems into a single embedded platform. The detailed objectives are as follows:

Objective 1: Design and Construction of a Functional Robot Chassis
The first objective is to build a stable, four-wheeled robot chassis using foam board and cardboard as the structural material. The chassis must securely house four DC gear motors, the Arduino UNO with motor driver shield, the sensor assembly, the Bluetooth module, and the battery pack in an organised and compact manner.

- **Target**: A rigid, lightweight chassis that can support all electronic components without mechanical failure during movement.
- **Purpose**: Four gear motors are glued to the foam board base; the motor shield is attached directly on top of the Arduino UNO and both are mounted centrally on the chassis; wires are routed through drilled holes for cleanliness.

Objective 2: Implementation of the Obstacle Avoidance System

The robot must be capable of autonomous navigation – moving forward freely, but detecting and avoiding obstacles without any human input.

- **Logic**: The HC-SR04 ultrasonic sensor continuously measures the distance to objects ahead. If the distance falls to ≤ 12 cm, the robot stops, moves backward briefly, and then scans left and right using the servo motor. It compares the free distances on both sides and turns towards the direction with greater clearance.
- **Purpose**: This objective demonstrates the principle of autonomous decision-making in embedded systems.

Objective 3: Implementation of Bluetooth Manual Control
The robot must accept and execute directional commands from a user's Android smartphone application via the HC-05 Bluetooth module.

- **Command Mapping**: `F` (Forward), `B` (Backward), `L` (Left), `R` (Right), `S` (Stop) – are transmitted from the app and received by the Arduino over the Serial port at **9600 baud rate**.
- **Purpose**: This objective demonstrates practical application of **UART serial communication** and **wireless data transfer**.

Objective 4: Implementation of Voice Control with Safety Logic

The robot must respond to spoken voice commands recognised by the smartphone application and transmitted as serial characters.

- **Voice Commands and Mapped Characters:**

Voice Command	Character Sent	Action
"Forward"	^	Move forward
"Backward"	-	Move backward
"Left"	<	Check left clearance, then turn
"Right"	>	Check right clearance, then turn
"Stop"	*	Stop all motors

Table 1.1: Voice Command Mapping

- **Safety Feature**: For the Left (<) and Right (>) commands, the robot first checks the respective side's clearance using `leftsee()` or `rightsee()` functions. If the measured distance is less than 10 cm, the robot does not execute the turn and instead stops – preventing collision.

Objective 5: Modular Program Design for Easy Mode Switching
All three functional modes must be implemented as separate, independent functions within a single Arduino sketch – `Obstacle()`, `Bluetoothcontrol()`, and `voicecontrol()`. Switching between modes requires only commenting or uncommenting a single line in the `loop()` function, making the code clean, maintainable, and user-friendly.

1.4 Scope and Applications

The scope of this project extends well beyond an academic demonstration. The principles and systems implemented in this robot car directly correspond to real-world technologies used across multiple industries.

Real-World Applications:

1. Automotive Safety Systems (Reverse Parking Sensors and ADAS)
The obstacle avoidance logic – where the robot stops and scans when an object is within 12 cm – directly mirrors the Automatic Emergency Braking (AEB) and Parking Assistance Systems found in modern automobiles. Vehicles such as those manufactured by Toyota, Honda, and Maruti Suzuki use similar ultrasonic arrays for proximity detection in tight parking situations.

2. Warehouse and Industrial Automation
In modern warehouses, Automated Guided Vehicles (AGVs) use sensor arrays and wireless communication to transport goods from one location to another without human intervention. The obstacle avoidance mode of this project represents the fundamental navigation logic employed in such industrial robots.

3. Assistive Robotics for the Differently Abled
Voice-controlled robotic systems are extensively used in assistive technology – for example, motorised wheelchairs that respond to voice commands, or robotic arms controlled by speech for individuals with limited mobility. The voice control mode of this project demonstrates the foundational principle of such systems.

4. Remote Surveillance and Security Monitoring
A Bluetooth-controlled robot car with a mounted camera module (a future upgrade) can be used for surveillance in hazardous or inaccessible areas – such as disaster zones, narrow pipelines, or military perimeters – where direct human entry poses a risk.

5. Educational Robotics and STEM Learning
Perhaps most importantly, this project serves as an ideal educational tool for students of Electrical Engineering, Electronics, and Computer Science. It covers a wide range of foundational concepts – DC motor control, PWM signals, serial communication, ultrasonic sensing, and wireless Bluetooth communication – all within a single, tangible, and engaging project.

Future Scope:
(Note: This is a brief mention here; detailed future scope is covered in Chapter 6)

The current system uses a wired USB connection for uploading code and two Li-ion batteries for power. Future enhancements include:

- Integration of an **ESP32** or **Wi-Fi module** for internet-based control from anywhere in the world (IoT).
- Addition of a **camera module (ESP32-CAM)** for live video feed to the smartphone.
- Development of a **custom Flutter-based Android application** *(RoboNex)* with joystick control, voice commands, radar animation display, and live distance monitoring – replacing the generic Bluetooth controller app.
- Upgrade to a **360-degree servo or stepper motor** for full-circle environmental scanning.

1.5 Organization of the Report

This project report is structured systematically to provide a comprehensive and logical understanding of the design, implementation, and analysis of the Multi-Function Robot Car. The report is divided into the following chapters:

Chapter 1: Introduction

This chapter provides an overview of the project, including the motivation behind building a multi-function robot, the specific problems it addresses, the detailed

objectives of each functional mode, and the scope of its real-world applications. It establishes the foundation for the entire project.

Chapter 2: Literature Review

This chapter explores the background of robotic control systems, comparing different wireless communication technologies (Bluetooth vs. Wi-Fi), reviewing existing robot control methods (IR, RF, Bluetooth, Voice), and justifying the selection of the L293D Motor Driver Shield over alternatives. It contextualises this project within the broader landscape of existing work.

Chapter 3: System Design and Hardware

This chapter details the complete hardware aspect of the project. It includes the system block diagram, individual descriptions and technical specifications of each component (Arduino UNO, L293D shield, HC-SR04, SG90 servo, HC-05 module, gear motors, and battery), the complete circuit diagram with pin connections, and the robot chassis construction procedure. This chapter serves as a complete guide for physically reproducing the project.

Chapter 4: Methodology and Software Implementation

This chapter explains the software design of the project. It describes the Arduino programming logic for each of the three modes, the distance calculation formula, the servo scanning mechanism, the voice command safety check logic, the system flowchart, and the method of mode switching. It bridges the hardware described in Chapter 3 with the working behaviour of the robot.

Chapter 5: Results and Analysis

This chapter presents the experimental outcomes and test results. It includes observation tables for obstacle avoidance distance accuracy, Bluetooth command response times, and voice recognition accuracy. It also provides an honest analysis of the system's limitations and sources of error.

Chapter 6: Conclusion and Future Scope

This final chapter summarises the project's success in meeting all five objectives. It also outlines potential future improvements – including wireless IoT integration, camera addition, and the development of the custom **RoboNex** Flutter Android application.

Chapter 7: References

This section lists all the sources, tutorials, datasheets, and research articles referred to during the course of this project.

Chapter 2 Literature Review

2.1 History and Evolution of Robotic Systems

The concept of a machine that could perform tasks autonomously – mimicking or replacing human effort – has fascinated engineers and scientists for centuries. However, the modern era of robotics truly began in the mid-20th century, with the development of the first programmable industrial robot.

Early Development:
The word "**Robot**" was first introduced by Czech playwright **Karel Čapek** in his 1920 science fiction play "R.U.R." (Rossum's Universal Robots), where it referred to artificial workers. The first truly programmable industrial robot, **Unimate**, was developed by **George Devol** in 1954 and was deployed on a General Motors assembly line in 1961. It was a fixed-arm robot designed to lift and stack hot metal parts – a task too dangerous for human workers. This marked the beginning of **industrial robotics**.

Evolution Through the Decades:

- **1960s–1970s**: Robots were large, fixed-arm machines used exclusively in heavy industry – automobile manufacturing, welding, and painting. They operated in strictly controlled, caged environments and had no sensing capability.
- **1980s–1990s**: The introduction of **microprocessors** miniaturised control electronics significantly. Robots became smaller, more precise, and began to incorporate basic sensors. The concept of **mobile robots** – machines that could move through an environment – emerged during this period. NASA's early planetary rover prototypes were developed in this era.
- **2000s**: The advent of **open-source microcontroller platforms**, most notably the **Arduino** (first released in 2005 at the Interaction Design Institute, Ivrea, Italy), democratised robotics entirely. For the first time, students and hobbyists without advanced electronics knowledge could build functional robots at extremely low cost. This triggered a global explosion in **educational and DIY robotics**.
- **2010s–Present**: Modern robots **integrate artificial intelligence, computer vision, wireless connectivity (Bluetooth, Wi-Fi, 5G), and voice recognition**. Robots such as Boston Dynamics' Spot, Amazon's warehouse robots, and autonomous vehicles demonstrate that the boundary between science fiction and engineering reality has effectively dissolved.

Relevance to This Project:
This project sits at the intersection of the educational robotics tradition pioneered by Arduino and the modern multi-modal control paradigm. By combining autonomous sensing, Bluetooth wireless control, and voice recognition into a single affordable platform, this project embodies the evolutionary trajectory of robotics – from single-function machines to adaptable, intelligent systems.

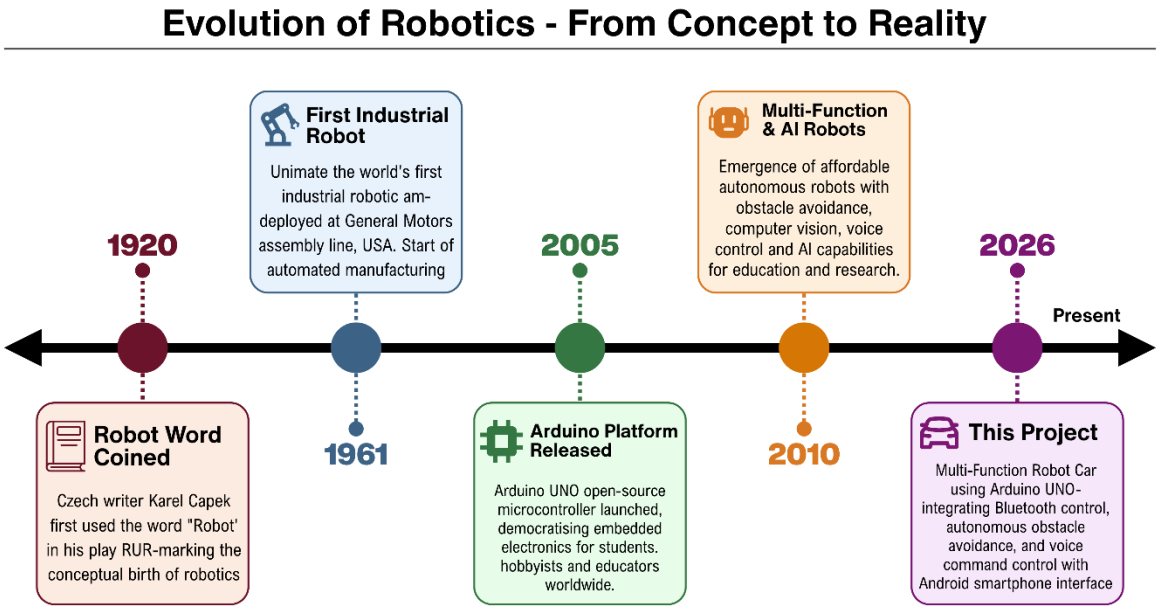


Figure 2.1 – Timeline of Robotics Evolution

2.2 Bluetooth vs. Wi-Fi Wireless Control

Wireless communication is the backbone of any remotely controlled robotic system. For this project, the selection of the appropriate wireless technology was a critical design decision. The two most commonly considered options for short-range robot control in embedded systems are Bluetooth and Wi-Fi. A thorough comparison of both technologies justifies the choice made in this project.

2.2.1 Bluetooth Communication

Bluetooth is a short-range wireless communication standard that operates in the 2.4 GHz ISM (Industrial, Scientific, and Medical) frequency band. It was originally developed by Ericsson in 1994 and is now governed by the Bluetooth Special Interest Group (SIG). The HC-05 module, used in this project, is a Bluetooth Classic (v2.0) module that communicates using UART (Universal Asynchronous Receiver-Transmitter) serial protocol

Key Characteristics of Bluetooth:

- Operating Frequency: **2.4 GHz**
- Effective Range: **10 metres** (Class 2, as in HC-05) up to 100 metres (Class 1)
- Power Consumption: **Very Low** (approx. 40 mA during transmission for HC-05)
- Data Transfer Rate: **Up to 3 Mbps** (Bluetooth Classic v2.0+EDR)
- Pairing: Requires one-time device pairing; no router or internet infrastructure needed
- Latency: **Very Low** (typically < 10 ms), making it ideal for real-time control applications
- Cost: **Extremely Low** (HC-05 modules available for ₹150–₹200)

2.2. 2 Wi-Fi Communication

Wi-Fi (IEEE 802.11 standard) is a medium-to-long range wireless networking technology also operating primarily in the **2.4 GHz and 5 GHz** bands. For embedded robotics, Wi-Fi is typically implemented using modules such as the **ESP8266** or **ESP32**.

Key Characteristics of Wi-Fi:

- Operating Frequency: **2.4 GHz / 5 GHz**
- Effective Range: **50–100 metres indoors** (varies by environment)
- Power Consumption: **Significantly Higher** (ESP8266 can draw up to 300 mA during transmission)
- Data Transfer Rate**: Up to 150 Mbps** (802.11n)
- Infrastructure: Requires a Wi-Fi router or Access Point; adds complexity
- Latency: **Moderate to High** (dependent on network conditions and router processing)
- Cost: **Moderate** (ESP8266 modules: ₹200–₹400)

2.2. 3 Comparative Summary

Feature	Bluetooth (HC-05)	Wi-Fi (ESP8266)
Operating Frequency	2.4 GHz	2.4 / 5 GHz
Range	~10 metres	~50–100 metres
Power Consumption	Very Low (~40 mA)	High (~300 mA)
Latency	Very Low (<10 ms)	Moderate
Infrastructure Required	None (Direct Pairing)	Requires Wi-Fi Router
Cost	₹150–₹200	₹200–₹400
Internet Capability	No	Yes
Ease of Use with Arduino	Very High	Moderate
Best Suited For	**Short-range real-time control**	**IoT and internet-based control**

Table 2.1: Comparison Between Bluetooth (HC-05) and Wi-Fi (ESP8266) for Robot Control

Conclusion of Comparison:
For this project, **Bluetooth (HC-05)** was selected over Wi-Fi for the following reasons:

1. The control range requirement is **within a classroom or small room** – well within Bluetooth's 10-metre range.
2. Bluetooth offers **significantly lower latency**, which is critical for responsive, real-time directional control.
3. The HC-05 module requires **no external router or network infrastructure** – it works as a direct point-to-point connection between the smartphone and the robot.

4. Bluetooth consumes **far less power**, which is essential given that this robot is battery-powered by only two Li-ion cells.
5. The HC-05 module communicates via **UART** serial protocol, which is natively supported by the Arduino's Serial.read() function – making programming straightforward.

2.3 Review of Existing Robot Control Methods

Before finalising the design of this robot, a thorough review of existing and commonly used robot control methods was conducted. Each method has distinct advantages and limitations that influence its suitability for different applications.

1. Infrared (IR) Sensors:
Infrared remote control uses IR light signals (wavelength: 700 nm – 1 mm) transmitted from a handheld remote to an IR receiver on the robot.

- ***Advantages***: Extremely cheap; simple to implement; no pairing required.
- ***Disadvantages***: Works only in **direct line-of-sight** – any obstruction between remote and robot breaks the signal. Susceptible to **interference from sunlight** and other IR sources (televisions, room heaters). Limited range (typically 5–10 metres). Cannot be integrated with a smartphone easily.
- ***Verdict***: *Unsuitable for this project* – lacks smartphone integration and voice control capability

2. Radio Frequency (RF) Control (433 MHz / 2.4 GHz)
RF control uses radio transmitter-receiver pairs to send directional commands to the robot.

- ***Advantages***: Works through walls; longer range than IR; does not require line-of-sight.
- ***Disadvantages***: Uses **dedicated RF transmitter hardware** (not a smartphone); cannot support voice commands; susceptible to frequency interference from other RF devices; implementation is more complex than Bluetooth.
- ***Verdict:*** *Partially suitable* – but does not fulfil the voice control and smartphone integration requirements.

3. Bluetooth Control (Selected Method)
As described in Section 2.2, Bluetooth provides low-latency, short-range, bidirectional wireless communication directly with a smartphone.

- ***Advantages***: Direct smartphone integration; supports both button-based and voice-based commands; low power; simple UART interface with Arduino.
- ***Disadvantages***: Limited to approximately 10-metre range.
- ***Verdict***: ***Best suited for this project***. Fulfils all control requirements – manual buttons, voice commands, and mode-switching signals.

4. Wi-Fi / IoT Control
As discussed in Section 2.2, Wi-Fi enables internet-based control over longer distances.

- ***Advantages***: Control from anywhere in the world via internet; supports camera streaming; higher data bandwidth.

- ***Disadvantages***: Higher power consumption; requires router infrastructure; higher latency for real-time control; more complex programming.
- ***Verdict***: ***Recommended as a Future Upgrade (see Chapter 6)*** – not selected for the current implementation due to power and complexity constraints.

5. Autonomous Sensor-Based Control (Obstacle Avoidance)
Autonomous control involves no human input – the robot uses on-board sensors to perceive and navigate its environment independently.

- **Advantages**: No operator required; suitable for hazardous environments; demonstrates AI-like decision-making.
- **Disadvantages**: Limited to pre-programmed logic; cannot be directed to a specific target; depends heavily on sensor accuracy
- **Verdict**: ***Implemented as Mode 1*** in this project – using HC-SR04 ultrasonic sensor with servo scanning.

Control Method	**Range**	**Smartphone Integration**	**Voice Support**	**Cost**	**Selected**
IR Remote	5–10 m (LOS only)	No	No	Very Low	✗
RF (433 MHz)	50–200 m	No	No	Low	✗
Bluetooth (HC-05)	**~10 m**	**Yes**	**Yes**	**Low**	✓
Wi-Fi (ESP8266)	50–100 m	Yes	Yes	Moderate	✗
Autonomous (Sensor)	N/A	N/A	N/A	Sensor Cost	✓

Table 2.2 – Comparative Review of Robot Control Methods

2.4 Comparison of Motor Drivers (Why L293D Shield?)

A **motor driver** is an essential intermediate circuit between a microcontroller and a DC motor. Microcontrollers like the Arduino UNO can supply only **40 mA** per digital I/O pin – far insufficient to drive even a small DC gear motor, which typically requires **100–300** mA. A motor driver acts as a **power amplifier** that uses a small control signal from the Arduino to switch a larger current from the battery to the motors.

For this project, the choice of motor driver was evaluated across three common options:

Option 1: L298N Motor Driver Module
The L298N is a dual H-Bridge motor driver IC mounted on a breakout board. It can drive two DC motors (or one stepper motor) with a peak output of 2A per channel.

- ***Advantages***: High current capacity (2A per channel); widely available; can drive larger motors.

- ***Disadvantages***: Drives only **two motors** directly – this project requires **four motors**, so two L298N modules would be needed. Consumes significant board space. Has a **voltage drop** of approximately 2V due to internal transistors, wasting battery power. Requires additional wiring for enable pins and motor direction pins.

Option 2: L293D IC (Bare Chip)
The L293D is a quadruple half-H driver IC – meaning it can independently control four DC motors or two stepper motors. It has built-in flyback diodes for motor protection.

- ***Advantages***: Controls four motors with a single IC; compact; low cost.
- ***Disadvantages***: When used as a bare chip on a breadboard, it requires **extensive manual wiring** – 16 pins to connect, multiple enable and direction pins to manage. This increases the risk of wiring errors and makes the circuit messy and difficult to debug.

Option 3: L293D Motor Driver Shield (Selected)
The **L293D Motor Driver Shield** (also known as the Adafruit Motor Shield V1 or AF Motor Shield) is a purpose-built Arduino shield that uses **two L293D ICs** on a single PCB, designed to **stack directly on top of the Arduino UNO**. It is controlled via the **AFMotor library** for Arduino.

- ***Advantages***:
  - Controls up to **four DC motors** (M1, M2, M3, M4) independently – perfectly matching this project's requirement
  - **Plugs directly onto the Arduino** – no breadboard wiring required; reduces connection errors dramatically.
  - Supports **PWM-based speed control** via `setSpeed()` function – easy to set motor speed from 0 to 255.
  - Supports **FORWARD**, **BACKWARD**, and **RELEASE** commands per motor via the `AFMotor` library.
  - Has a **dedicated power terminal** for external battery supply to the motors – separating motor power from Arduino logic power, preventing voltage drops from affecting the microcontroller.
  - Built-in flyback diodes protect the circuit from voltage spikes generated by motor switching.
- ***Disadvantages:*** *Maximum current is* ***0.6A continuous / 1.2A peak per channel –*** *sufficient for small gear motors but not for high-torque motor*s.

Feature	**L298N Module**	**L293D IC (Bare)**	**L293D Shield (Selected)**
No. of DC Motors Controlled	2	4	**4**
Current Per Channel	2A	0.6A	**0.6A (1.2A peak)**

Arduino Compatibility	External wiring	Manual wiring	**Direct stack (plug-in)**
Wiring Complexity	High	Very High	**None (plug-in)**
Library Support	Manual PWM	Manual PWM	**AFMotor (simple)**
Flyback Diode Protection	Yes	Yes	**Yes**
Board Space Required	Moderate	Large (breadboard)	**None (on Arduino)**
Cost	₹100–₹150	₹30–₹50 (IC only)	**₹150–₹250**
Best For	2-motor robots	Manual builds	**4-motor shield robots**

Table 2.3 – Comparison of Motor Driver Options

Conclusion:
The **L293D Motor Driver Shield** was selected as the motor driving solution for this project because it is the **only option** that simultaneously satisfies all four requirements: driving four motors, simple integration with Arduino, software-friendly control via the `AFMotor` library, and a clean, compact build. Its plug-and-play design eliminates the most common source of hardware errors in student projects – incorrect motor driver wiring.

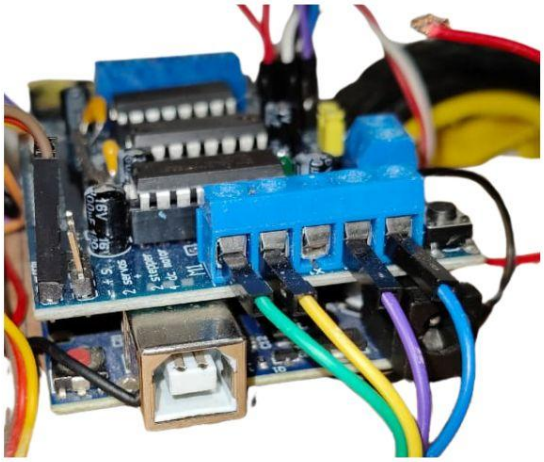


Figure 2.2 – L293D Motor Driver Shield mounted on Arduino UNO

Chapter 3 System Design and Hardware

3.1 Block Diagram of the Proposed System

The "**MULTI-FUNCTION ROBOT CAR USING ARDUINO UNO**" is composed of three major subsystems working in coordination: the Input Stage, the Processing Stage, and the Output Stage. Understanding the data flow between these stages is essential before examining the individual hardware components.

Data Flow Description:

1. **Input Stage:** Two types of inputs feed into the system:
   - ☞ The **HC-SR04 Ultrasonic Sensor** continuously measures the distance of objects in front of (or to the sides of) the robot and sends a digital echo signal to the Arduino.
   - ☞ The **HC-05 Bluetooth Module** receives wireless serial data from the Android smartphone – either button commands (`F, B, L, R, S`) in Bluetooth mode or voice-converted characters (`^, -, <, >, *`) in Voice Control mode.

2. **Processing Stage**: The **Arduino UNO** is the brain of the entire system. It:
   - ☞ Reads the ultrasonic echo duration and calculates distance in centimetres.
   - ☞ Reads incoming Bluetooth serial characters and identifies the active command.
   - ☞ Executes the appropriate function: `Obstacle(), Bluetoothcontrol()`, or `voicecontrol()`.
   - ☞ Generates PWM signals to control the L293D Motor Shield and position commands to control the SG90 Servo Motor.

3. **The Output (Hardware):**
   - ☞ The **L293D Motor Driver Shield** receives control signals from the Arduino and drives the four DC gear motors (M1, M2, M3, M4) accordingly – Forward, Backward, Left, Right, or Stop.
   - ☞ The **SG90 Servo Motor** rotates the ultrasonic sensor to 180° (left scan) or 20° (right scan) during obstacle avoidance and voice-controlled turns.

4. **Output Stage (User Feedback):**
   1. The **Android Smartphone App** displays the connection status (green indicator when connected) and reflects the active control mode.

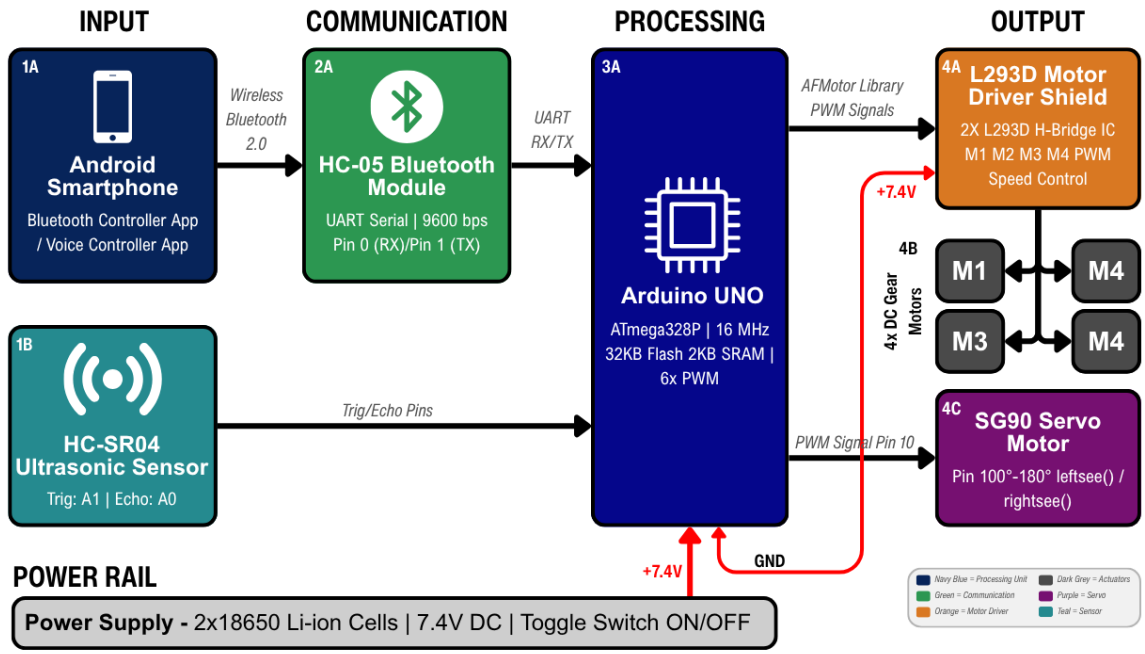
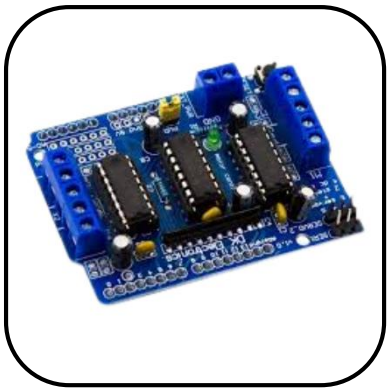


Figure 3.1 – Block Diagram of the Multi-Function Robot Car System

3.2 Hardware Components Description

This project utilises eight categories of hardware components, each selected based on their technical suitability, cost-effectiveness, and compatibility with the Arduino platform. Below is a detailed technical description of each component.

1. Arduino UNO R3 ATmega328P



2. L293D Motor Driver Shield

3. HC-SR04 Ultrasonic Sensor

4. SG90 Servo Motor

5. HC-05 Bluetooth Module

6. DC Gear Motors with Wheel

7. Li-ion Battery with Holder

8. Jumper wires

9. Foam/Card board

3.2.1 Arduino UNO (Microcontroller Architecture)

The **Arduino UNO** is the central processing unit of the entire robot car system. It is an open-source microcontroller development board based on the **ATmega328P 8-bit** AVR microcontroller chip, manufactured by Microchip Technology.

Role in Project:

The Arduino UNO generates trigger pulses for the ultrasonic sensor, reads the echo duration, calculates distances, reads incoming Bluetooth serial data, executes the appropriate mode function, sends PWM signals to the L293D Motor Shield for motor control, and sends PWM position signals to the SG90 servo motor.

Table 3.1 - Technical Specifications of Arduino UNO:

Parameter	**Specification**
Microcontroller	ATmega328P (8-bit AVR)
Clock Speed	16 MHz
Operating Voltage	5V DC
Input Voltage (Recommended)	7–12V (DC Jack) / 5V (USB)
Digital I/O Pins	14 (of which 6 support PWM)
Analog Input Pins	6 (A0–A5)
Flash Memory	32 KB (0.5 KB used by bootloader)
SRAM	2 KB
EEPROM	1 KB
UART Serial	1 (Pins 0/RX and 1/TX)
USB Interface	ATmega16U2 USB-to-Serial Converter

Why Arduino UNO was selected:

1. Its **16 MHz clock speed** provides sufficient processing power for simultaneous sensor reading, Bluetooth communication, and motor control.
2. The built-in **USB-to-Serial converter** (ATmega16U2) allows direct code upload and Serial Monitor access without any external programmer.
3. The **L293D Motor Shield stacks directly on top** of the Arduino UNO's standard pin headers – no additional wiring needed.
4. The vast **Arduino ecosystem** (libraries, community support, tutorials) makes implementation and debugging straightforward.

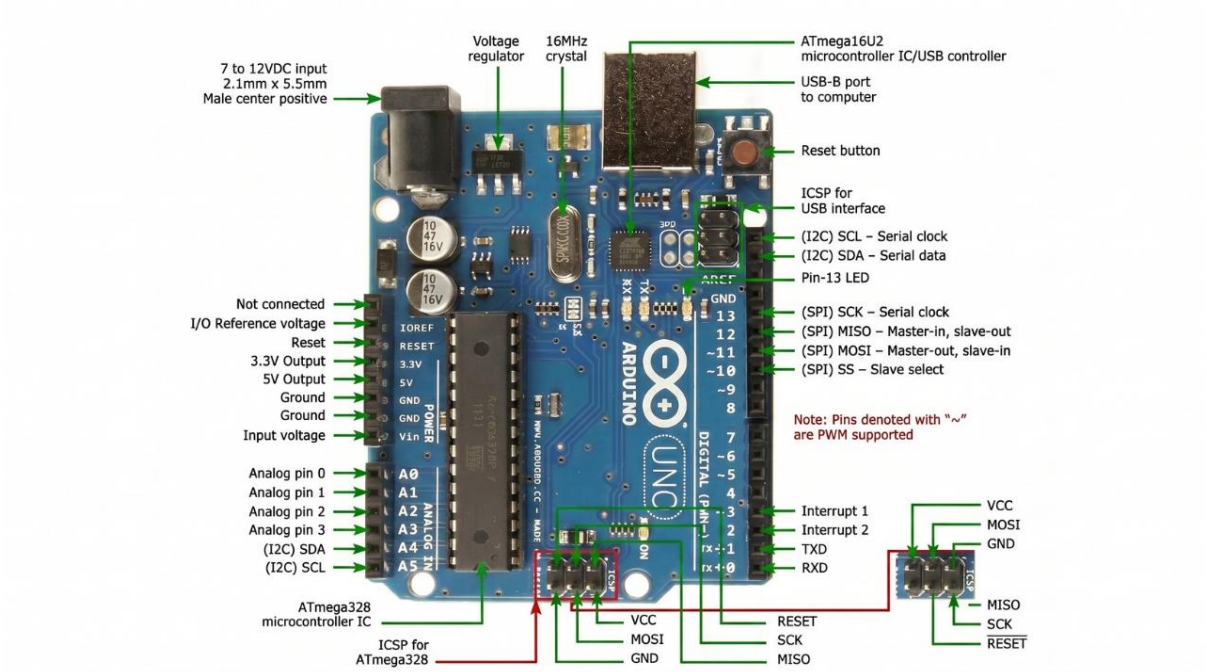


Figure 3.2 – Arduino UNO Board with Pin Labels

3.2.2 L293D Motor Driver Shield (Working Principle)

The **L293D Motor Driver Shield** (also known as the *Adafruit Motor Shield V1 or AF Motor Shield*) is a purpose-built Arduino expansion board that integrates two **L293D H-Bridge ICs** on a single PCB. It is designed to stack directly on top of the Arduino UNO via standard pin headers.

What is an H-Bridge?

An **H-Bridge** is an electronic circuit consisting of four switching elements (transistors) arranged in an "H" shape. By selectively turning pairs of transistors ON and OFF, the circuit can reverse the direction of current flow through a motor – enabling both **FORWARD** and **BACKWARD** rotation from a single power supply. The L293D IC contains two independent

H-Bridge circuits, and since the shield uses two L293D ICs, it can independently control **four separate DC motors**.

Role in This Project:

The Motor Shield receives direction and speed commands from the Arduino (via the AFMotor library) and drives all four **DC gear motors** (M1, M2, M3, M4) that propel the robot.

Key Technical Specifications:

Parameter	**Specification**
Motor ICs Used	2 × L293D (Quad Half-H Driver)
Number of DC Motors Supported	4 (M1, M2, M3, M4)
Continuous Current Per Channel	0.6A
Peak Current Per Channel	1.2A
Motor Supply Voltage	4.5V – 25V DC (External Terminal)
Logic Supply Voltage	5V (from Arduino)
PWM Speed Control	Yes (via AFMotor library, 0–255)
Motor Directions Supported	FORWARD, BACKWARD, RELEASE
Built-in Flyback Diodes	Yes (motor spike protection)
Interface with Arduino	Direct plug-in (stacks on UNO)

Table 3.2 – Technical Specifications of L293D Motor Driver Shield

Flyback Diode Protection:

DC motors are inductive loads – when their current is suddenly cut (motor stops), they generate a reverse-polarity voltage spike that can damage the microcontroller. The L293D shield's **built-in flyback diodes** safely redirect these spikes to the power rail, protecting the Arduino UNO.

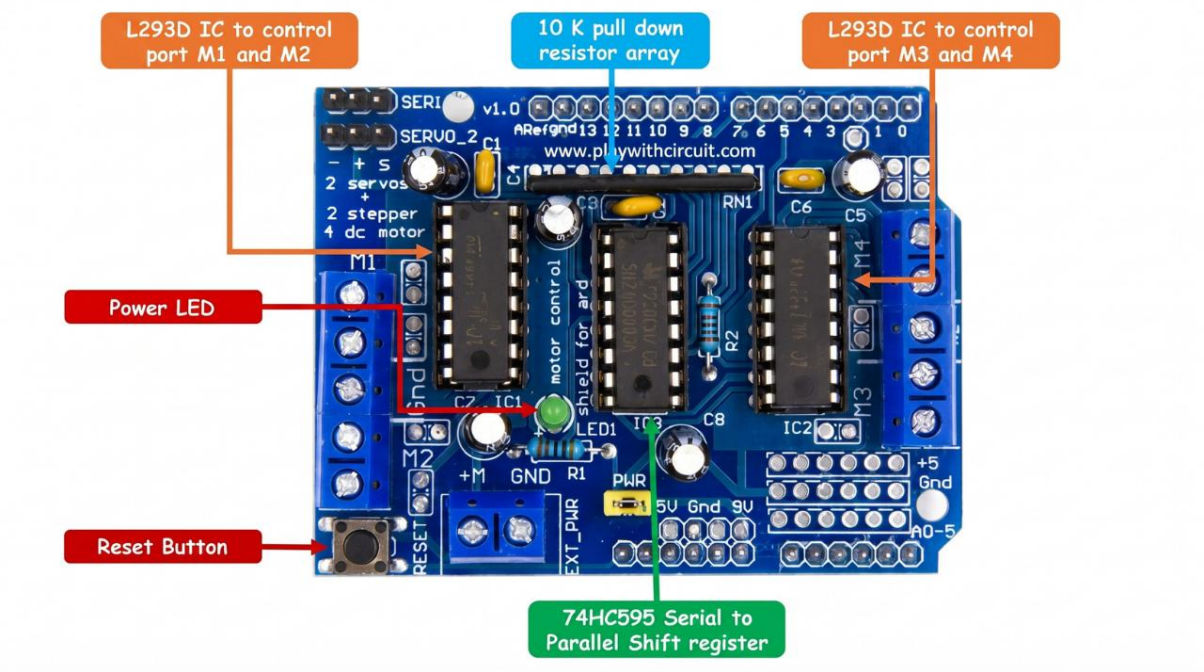


Figure 3.3 – L293D Motor Driver Shield

3.2.3 HC-SR04 Ultrasonic Sensor (Working Principle)

The **HC-SR04** is a non-contact distance measurement module that uses **ultrasonic sound waves** to detect objects and measure their distance. It is one of the most widely used sensors in educational robotics due to its accuracy, reliability, and extremely low cost.

Working Principle – Time of Flight (ToF):

The HC-SR04 operates on the **time-of-flight principle**. The sensor consists of two cylindrical transducers – a **Transmitter (T)** that functions like a miniature loudspeaker and a **Receiver (R)** that functions like a microphone.

The measurement cycle proceeds as follows:

1. The Arduino sends a **10-microsecond HIGH** pulse to the **TRIG pin** (A1 in this project).
2. This triggers the transmitter to emit a burst of **8 ultrasonic pulses at 40 kHz** frequency – well above the human hearing range (20 Hz – 20 kHz).
3. These sound waves travel through the air at approximately **343 metres per second** and bounce off any object in their path.
4. The reflected echo returns to the receiver. The sensor immediately outputs a **HIGH signal** on the **ECHO pin** (A0 in this project).
5. The duration of this HIGH pulse – measured by the Arduino using the pulseIn() function – is directly proportional to the distance of the object.

Distance Calculation:

$$Distance\ (cm) = \frac{t}{29\ \times 2}$$

Where t = duration of ECHO HIGH pulse in microseconds.

Derivation: Speed of sound ≈ 343 m/s = 0.0343 cm/µs ≈ 1/29.2 cm/µs. Since the pulse travels to the object AND back (twice the distance), the result is divided by 2.

Role in Project:

- In **Obstacle Avoidance Mode**: Continuously measures forward distance; triggers stop-and-scan logic when ≤ 12 cm.
- In **Voice Control Mode**: Checks left or right side clearance (via servo positioning) before executing a left/right turn; prevents collision if clearance < 10 cm.

Key Technical Specifications:

Parameter	**Specification**
Operating Voltage	5V DC
Operating Current	15 mA (active) / < 2 mA (standby)
Operating Frequency	40 kHz
Measuring Range	2 cm – 400 cm
Measuring Angle	~15° (cone angle)
Resolution	~3 mm
Trigger Input Signal	10 µs HIGH TTL pulse
Echo Output Signal	HIGH TTL pulse (duration = distance)
Number of Pins	4 (VCC, TRIG, ECHO, GND)

Table 3.3 – Technical Specifications of HC-SR04 Ultrasonic Sensor

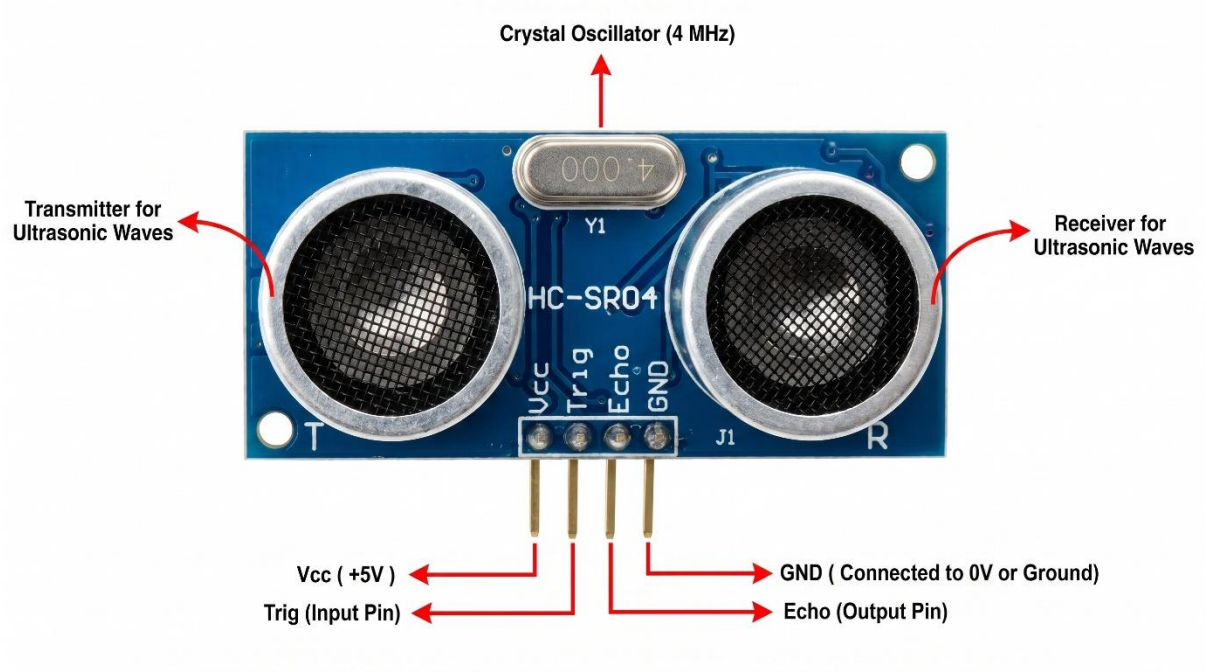


Figure 3.4 – HC-SR04 Ultrasonic Sensor with Transmitter and Receiver Labelled

3.2.4 SG90 Servo Motor (Mechanism and Control)

The **SG90** is a miniature, lightweight servo motor capable of precise angular positioning. Unlike a standard DC motor that rotates continuously, a servo motor rotates to – and holds – a **specific angle** commanded by a PWM signal.

Working Principle:

The SG90 contains a small **DC motor, a plastic gear train, and a feedback potentiometer** – all enclosed within a compact plastic housing. The potentiometer is mechanically linked to the output shaft and electrically feeds the current shaft position to an internal control circuit. When the Arduino sends a **PWM signal** specifying a target angle, the internal circuit compares the target with the current position and drives the motor until the shaft reaches the commanded angle. This is called a **closed-loop control system**.

PWM Control:

The SG90 is controlled by a PWM signal with a period of 20 ms (50 Hz). The pulse width determines the angle:

- ☞ 1 ms pulse width → 0° position
- ☞ 1.5 ms pulse width → 90° (centre) position
- ☞ 2 ms pulse width → 180° position

The Arduino's Servo.h library abstracts this entirely – `servo.write(angle)` is all that is required.

Role in Project:

- **Centre/Default Position**: `servo.write(103)` – the sensor points straight ahead (slightly right of centre due to physical mounting). The value `spoint = 103` is defined in the code.
- **Left Scan**: `servo.write(180)` – sensor points left; `leftsee()` function measures left clearance.
- **Right Scan**: `servo.write(20)` – sensor points right; `rightsee()` function measures right clearance.

Key Technical Specifications:

Parameter	**Specification**
Operating Voltage	4.8V – 6.0V DC
Stall Torque (4.8V)	1.8 kg·cm (25 oz·in)
Operating Speed (4.8V)	0.12 sec / 60° (no load)
Rotation Range	0° – 180°
Control Signal	PWM (50 Hz, 1–2 ms pulse)
Weight	~9 grams
Gear Type	Plastic (Nylon / POM)
Connector	3-wire (Brown = GND, Red = VCC, Orange = Signal)

Table 3.4 – Technical Specifications of SG90 Servo Motor

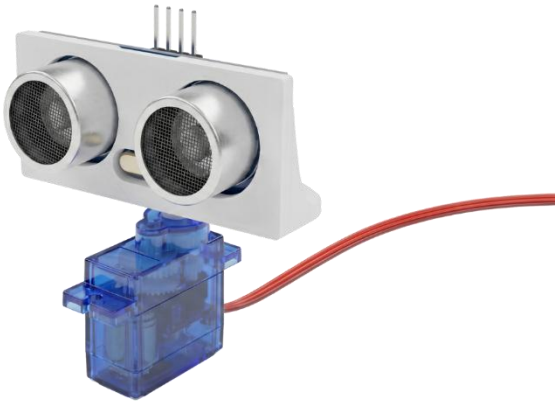


Figure 3.5 – SG90 Servo Motor with Ultrasonic Sensor mounted on Servo Horn

3.2.5 HC-05 Bluetooth Module (Communication)

The **HC-05** is a **Bluetooth Classic (v2.0) Serial Port Protocol (SPP)** module that enables wireless serial communication between the Arduino and an Android smartphone. It operates in the **2.4 GHz ISM band** and uses **UART communication** to interface with the Arduino.

Working Principle:

The HC-05 contains a **CSR BC417 Bluetooth chip** with an integrated antenna. When paired with an Android smartphone, it creates a transparent **virtual serial port** – data sent from the smartphone app appears as standard `Serial.read()` data on the Arduino, and data written via `Serial.print()` on the Arduino appears on the smartphone app. This transparent bridge is what makes integration with the Arduino's existing `Serial` functions so seamless.

Operating Modes:

- ☞ **AT Command Mode**: Accessed by holding the button on the module during power-up; used to configure baud rate, device name, and PIN. Default name: `HC-05`; Default PIN: `1234`.
- ☞ **Data Mode (Normal)**: Default operating mode after pairing; transparent serial bridge.

Role in This Project:

- ➢ Receives button commands from the Bluetooth Controller App: `F`, `B`, `L`, `R`, `S`.
- ➢ Receives voice-converted commands from the Voice Controller App: `^`, `-`, `<`, `>`, `*`.
- ➢ All received characters are read by `Serial.read()` in the Arduino and processed by `Bluetoothcontrol()` or `voicecontrol()` functions.

Key Technical Specifications:

Parameter	**Specification**
Bluetooth Standard	v2.0 + EDR
Frequency Band	2.4 GHz ISM
Effective Range	~10 metres (Class 2)
Operating Voltage	3.3V – 5V DC (onboard regulator)
Operating Current	~40 mA (transmitting) / 8 mA (standby)
Default Baud Rate	9600 bps

Default Device Name	HC-05
Default PIN	1234
Interface	UART (TX, RX pins)
Antenna	Built-in PCB trace antenna

Table 3.5 – Technical Specifications of HC-05 Bluetooth Module

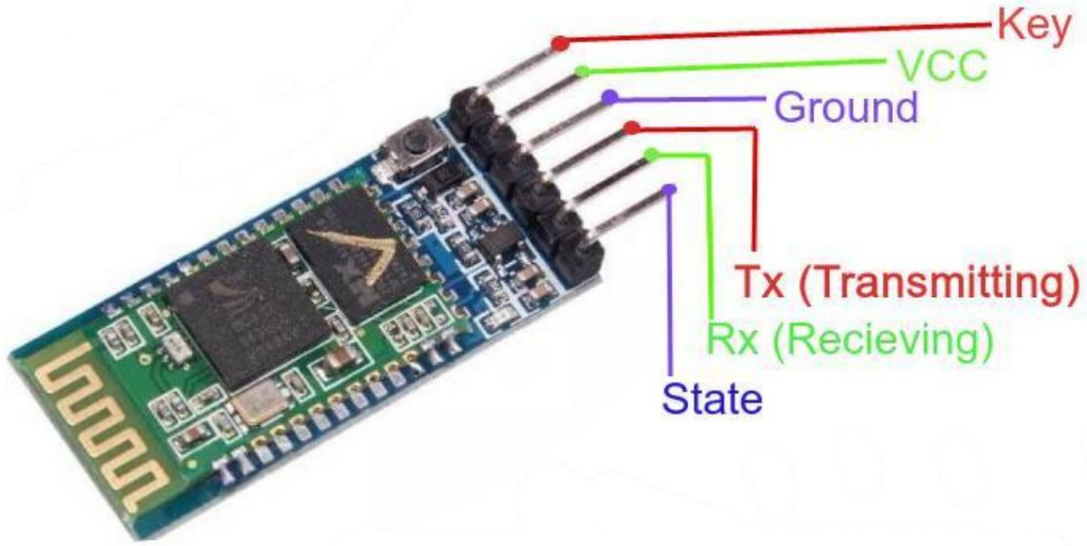


Figure 3.6 – HC-05 Bluetooth Module showing all Pins

3.2.6 DC Gear Motors and Robot Wheels

DC Gear Motors:

The robot uses **four identical TT DC Gear Motors** – a standard in Arduino-based robot car kits. These are small, brushed DC motors coupled to a **plastic gear reduction box** (typically 1:48 gear ratio), which reduces the RPM but **increases the output torque**, making the motor suitable for driving the wheels of a loaded chassis.

- **Operating Voltage**: 3V – 6V DC
- **No-Load Speed (6V):** ~200 RPM
- **Stall Torque:** ~800 g·cm
- **Current Draw (6V, no-load):** ~100–150 mA per motor

In this project, all four motors are set to the same speed via `M1.setSpeed(170)` through `M4.setSpeed(170)` – where 170 is a PWM value out of 255, resulting in approximately **67% of maximum speed** for smooth, controllable movement.

Motor Directional Logic (as implemented in code):

Movement	**M1**	**M2**	**M3**	**M4**
Forward	FORWARD	FORWARD	FORWARD	FORWARD
Backward	BACKWARD	BACKWARD	BACKWARD	BACKWARD
Turn Left	FORWARD	FORWARD	BACKWARD	BACKWARD
Turn Right	BACKWARD	BACKWARD	FORWARD	FORWARD
Stop	RELEASE	RELEASE	RELEASE	RELEASE

Table 3.6 – Motor Direction Truth Table for Robot Movements

Figure 3.7 – DC Gear Motor

Robot Wheels:

Four 65 mm diameter rubber wheels with a yellow plastic hub are used. The rubber tyre provides friction on most surfaces and the wheel diameter is appropriately matched to the gear motor's shaft diameter for direct press-fit attachment.

Figure 3.8 – DC Gear Motor with wheel attached

3.2.7 Li-ion Battery and Power Supply

The robot is powered by **two 3.7V Li-ion 18650 batteries** connected in series through a **dual-cell Li-ion battery holder**. Two cells in series produce a **nominal voltage of 7.4V** (fully charged: ~8.4V).

Power Distribution:

The battery holder output is connected directly to the external power terminal of the L293D Motor Shield. The Motor Shield routes this voltage in two paths:

1. **Motor Power Rail (VM):** 7.4V goes directly to the L293D H-Bridge circuits to drive the four DC motors.
2. **Arduino Power (VCC):** The Motor Shield also feeds the 7.4V through the Arduino's **VIN pin**, which routes through the Arduino's onboard **7805-compatible linear voltage regulator**, stepping it down to **5V DC** for the Arduino's logic circuits, and subsequently powering the HC-SR04 sensor, SG90 servo, and HC-05 module from the Arduino's 5V rail.

Power Budget Analysis:

Component	**Typical Current Draw**
Arduino UNO (logic)	~50 mA
HC-SR04 Ultrasonic Sensor	~15 mA
SG90 Servo Motor (moving)	~100–250 mA
HC-05 Bluetooth Module	~40 mA
4 × DC Gear Motors (loaded)	~400–600 mA total

Estimated Total Peak Current	**~700–950 mA**

Table 3.7 – Power Consumption Estimate of the Robot System

Two 18650 Li-ion batteries with a typical capacity of 2000–3000 mAh each provide ample power for 2–4 hours of continuous operation depending on load conditions.

Figure 3.9 – Li-ion Battery Holder with Two 18650 Cells Installed

3.3 Circuit Diagram and Pin Connections

The circuit for the Multi-Function Robot Car integrates all the components described in Section 3.2 through the L293D Motor Shield – which serves as the central connection hub for this project.

Circuit Description:

The L293D Motor Shield plugs directly onto the Arduino UNO. All motor connections are made directly to the **M1, M2, M3, M4 screw terminals** on the shield. The ultrasonic sensor and servo motor connect to the Arduino's **analog and digital pins** which are accessible through the shield's pass-through headers. The HC-05 Bluetooth module connects to the Arduino's **hardware serial pins (RX/TX)** via the shield.

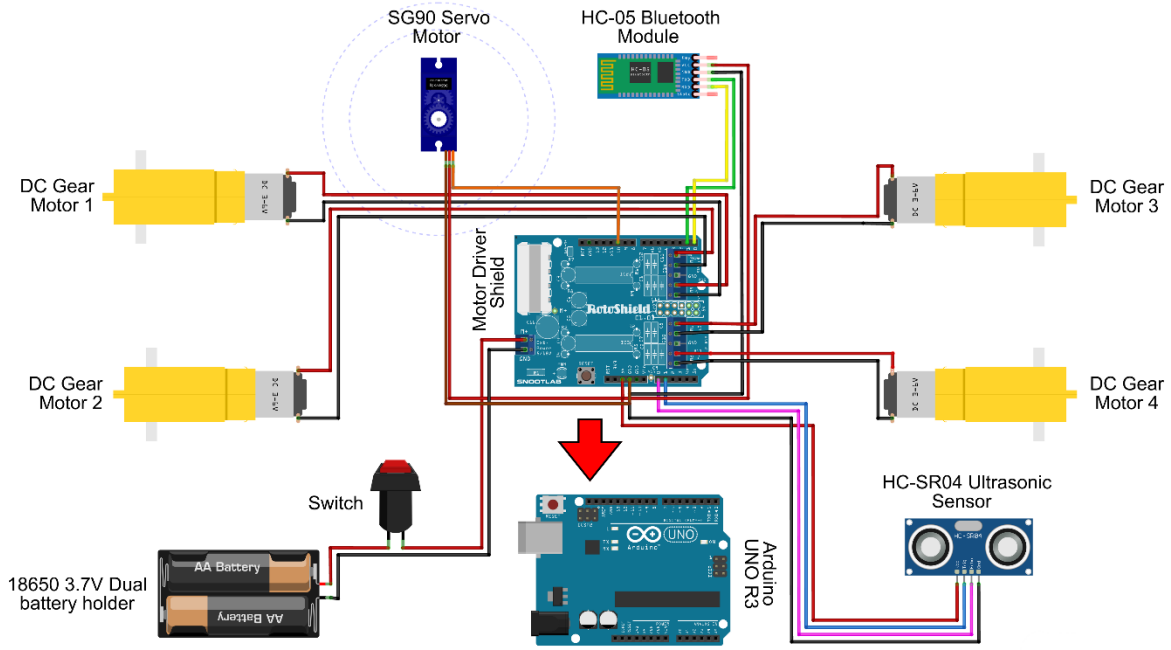


Figure 3.10 – Complete Circuit Diagram of the Robot Car System

Complete Pin Configuration Table:

Component	Component Pin	Wire Colour	Arduino / Shield Pin	Pin Type
HC-SR04 Sensor	VCC	Red	5V	Power
	GND	Black	GND	Ground
	TRIG	Yellow	A1	Digital Output
	ECHO	Green	A0	Digital Input
SG90 Servo Motor	VCC (Red)	Red	5V	Power
	GND (Brown)	Black	GND	Ground
	Signal (Orange)	Orange	Pin 10	PWM Output
HC-05 Bluetooth	VCC	Red	5V	Power
	GND	Black	GND	Ground
	TX	Green	Pin 0 (RX)	Serial Input
	RX	Blue	Pin 1 (TX)	Serial Output
DC Motor M1	Motor +/−	–	M1 Terminal (Shield)	Motor Output

DC Motor M2	Motor +/−	–	M2 Terminal (Shield)	Motor Output
DC Motor M3	Motor +/−	–	M3 Terminal (Shield)	Motor Output
DC Motor M4	Motor +/−	–	M4 Terminal (Shield)	Motor Output
Battery Pack	Positive (+)	Red	Shield Power Terminal (+)	Power Input
	Negative (−)	Black	Shield Power Terminal (−)	Power Input

Table 3.8 – Complete Pin Configuration and Wiring Details

Important Wiring Note:
When uploading code to the Arduino via USB, the **TX and RX wires of the HC-05 module must be temporarily disconnected** from Arduino Pins 0 and 1. This is because the Arduino uses these same pins for USB serial communication during code upload – having the HC-05 connected simultaneously causes **upload conflicts and errors**. After successful upload, reconnect the HC-05 wires.

3.4 Robot Chassis Construction

The chassis of the robot is constructed using **foam board and cardboard** – materials chosen for their lightness, ease of cutting, and sufficient structural rigidity for this project's weight requirements. This approach makes the chassis entirely reproducible using simple workshop tools.

Construction Steps:

1. **Cutting the Base Plate**: Cut a rectangular piece of foam board to serve as the main chassis base. The dimensions should be sufficient to accommodate all four gear motors (two on each side), the Arduino + Motor Shield assembly in the centre, and the battery holder at the rear.
2. **Mounting the Gear Motors**: Glue the four DC gear motors to the underside of the foam board – two on the left side (M1, M2) and two on the right side (M3, M4). Ensure the motor shafts extend outward beyond the edges of the foam board so the wheels can be attached freely.
3. **Mounting the Arduino and Motor Shield**: First, attach the L293D Motor Shield on top of the Arduino UNO by aligning and pressing the shield pins into the Arduino headers. Then, glue the combined Arduino + Shield assembly to the centre-top surface of the foam board chassis. This central placement distributes weight evenly.
4. **Cable Routing**: Drill or cut two small holes on either side of the Arduino mounting area. Thread the gear motor wires through these holes from the underside to the top side of the chassis. Connect each motor's wires to the corresponding M1, M2, M3, M4 screw terminals on the Motor Shield.

5. **Mounting the Servo and Ultrasonic Sensor Assembly**: Glue the SG90 servo motor to the front-centre of the chassis, with the servo shaft facing upward. Mount (glue) the HC-SR04 ultrasonic sensor firmly onto the servo horn so that it faces forward. Connect the servo signal wire to Pin 10 and the sensor wires to A0, A1 as per Table 3.8.
6. **Mounting the Bluetooth Module**: Glue the HC-05 Bluetooth module to the side or top of the chassis at a convenient position. Connect its TX and RX wires to Arduino Pins 0 and 1 respectively (remembering to disconnect during code upload).
7. **Mounting the Battery Holder**: Glue the dual Li-ion battery holder to the rear of the chassis for balanced weight distribution. Connect the battery holder output wires to the Motor Shield's external power terminal.
8. **Attaching the Wheels**: Press-fit the four 65 mm rubber wheels onto the motor shafts. Insert the Li-ion batteries into the holder.

Figure 3.11 – Completed Robot Car – Top View showing all components mounted

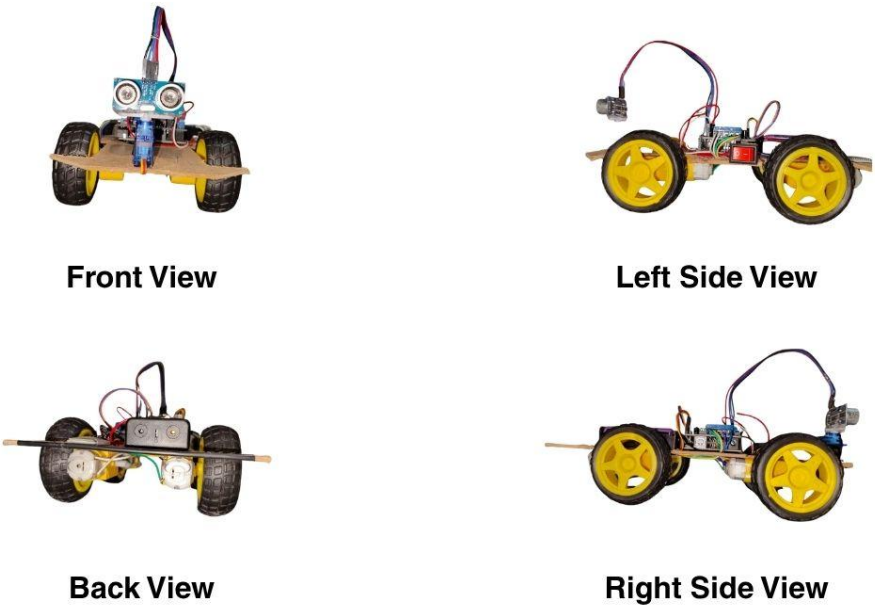


Figure 3.12 – Completed Robot Car — Front/Side View

Chapter 4 Methodology and Software Implementation

The methodology of the "**Multi-Function Robot Car Using Arduino UNO**" project is divided into two primary components: the **software tools** used for programming and control, and the **Arduino programming logic** that governs the behaviour of each operational mode. This chapter presents a detailed, step-by-step explanation of the complete software implementation – from library inclusion to individual function logic – followed by the system flowchart and the mode switching procedure.

4.1 Software Tools Used

4.1.1 Arduino IDE

The Arduino Integrated Development Environment (IDE) is the official software platform used to write, compile, and upload code to the Arduino UNO microcontroller. It is a free, open-source application available for Windows, macOS, and Linux operating systems, developed and maintained by Arduino S.r.l.

Key Features of the Arduino IDE used in this project:

- **Code Editor**: A syntax-highlighted text editor where the Arduino sketch (`.ino` file) is written in a simplified version of **C++.**
- **Compiler**: Based on the avr-gcc compiler toolchain; converts the human-readable sketch into machine-level binary (HEX) file.
- **Uploader (avrdude):** Transfers the compiled HEX file to the Arduino UNO's ATmega328P flash memory via the USB serial connection.
- **Serial Monitor**: A built-in terminal window for viewing data printed by `Serial.println()` during debugging. In this project, the Serial Monitor was used to verify incoming Bluetooth character values during testing.
- **Library Manager**: Allows one-click installation of third-party libraries. Both `Servo.h` and `AFMotor.h` were added through the Library Manager.

Board and Port Configuration used in this project:

- **Board:** Arduino UNO
- **Processor:** ATmega328P
- **Port:** COM port (assigned by Windows Device Manager when Arduino is connected via USB)
- **Baud Rate (Serial Monitor):** 9600 bps – matching Serial.begin(9600) in the code

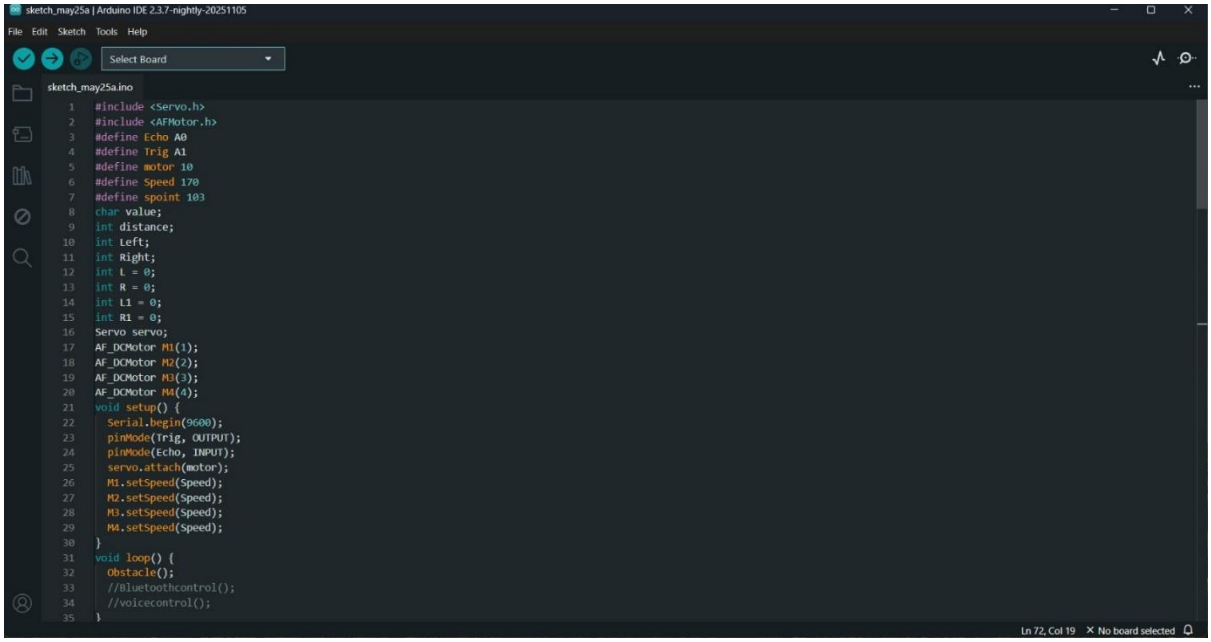


Figure 4.1 – Screenshot of Arduino IDE Interface

4.1.2 Bluetooth Controller Android App

Since this project uses the HC-05 Bluetooth Classic module, a compatible Android application is required on the smartphone to send control commands wirelessly. Two separate app configurations are used – one for Bluetooth manual control and one for voice control.

App Used: RoboNex / Bluetooth RC Controller / Arduino Bluetooth Controller

(A commonly available free application on the Google Play Store designed for HC-05/HC-06 based Arduino robot cars)

How the App Functions:

1. The user opens the app and navigates to **Settings → Connect to Car**.
2. The app scans for available Bluetooth devices and displays the **HC-05** module by its name (`HC-05`).
3. The user selects the device and enters the pairing PIN (`1234`).
4. Upon successful connection, the app displays a **green indicator** confirming active Bluetooth link.
5. For **Bluetooth Control Mode**: The app presents directional buttons (Forward, Backward, Left, Right, Stop). Each button, when pressed, transmits the corresponding single character over Bluetooth.
6. For **Voice Control Mode**: The user navigates to Settings → Voice Commands Configuration and maps each voice command word to its corresponding transmit character:

Voice Word Spoken	Character Transmitted to Arduino
"Forward"	^
"Backward"	–
"Left"	<
"Right"	>
"Stop"	*

Table 4.1 – Voice Command Configuration in the Android App

After configuration, the user presses the **Voice Control Button** in the app, speaks a command, and the app's **speech recognition engine** (Google Speech-to-Text API) converts the spoken word to text, matches it against the configured list, and transmits the corresponding character via Bluetooth to the HC-05 module.

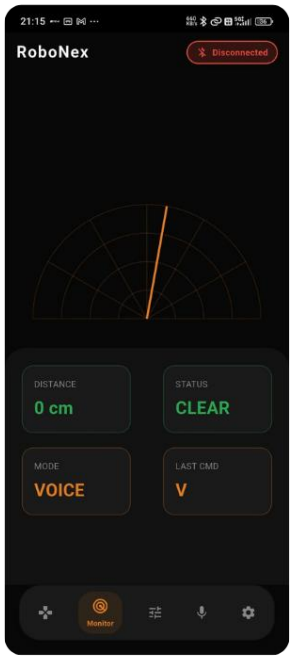


Figure 4.2 – Screenshots of Bluetooth Controller App

4.2 Arduino Programming Logic

The complete Arduino program for this project is written as a single unified sketch containing all three operational mode functions. The program uses a modular structure where each mode is encapsulated in its own function, and the `loop()` function calls only the active mode.

Global Definitions and Variable Declarations:

```
#include <Servo.h>
#include <AFMotor.h>

#define Echo A0          // Ultrasonic ECHO pin
#define Trig A1          // Ultrasonic TRIG pin
#define motor 10         // Servo motor signal pin
#define Speed 170        // Motor speed (PWM: 0–255)
#define spoint 103       // Servo centre/default position (degrees)

char value;              // Stores incoming Bluetooth character
int distance;            // Stores calculated distance in cm
int Left, Right;         // Stores left/right scan distances
int L = 0, R = 0;        // Working variables for obstacle logic
int L1 = 0, R1 = 0;      // Reserved variables

Servo servo;             // Servo object (Servo.h library)
AF_DCMotor M1(1);        // Motor 1 object (AFMotor library)
AF_DCMotor M2(2);        // Motor 2 object
AF_DCMotor M3(3);        // Motor 3 object
AF_DCMotor M4(4);        // Motor 4 object
```

Setup Function:

```
void setup() {
  Serial.begin(9600);        // Initialise Serial at 9600 baud (HC-05 default)
  pinMode(Trig, OUTPUT);     // TRIG pin set as output
  pinMode(Echo, INPUT);      // ECHO pin set as input
  servo.attach(motor);       // Attach servo to Pin 10
  M1.setSpeed(Speed);        // Set all motors to Speed = 170
  M2.setSpeed(Speed);
  M3.setSpeed(Speed);
  M4.setSpeed(Speed);

}
```

4.2.1 Libraries Used (`Servo.h` and `AFMotor.h`)

Two external libraries form the software foundation of this project:

1. Servo.h

The `Servo.h` library is a **standard Arduino built-in library** that provides simple control of servo motors via PWM signals. It abstracts the complex PWM timing entirely, allowing the programmer to simply call `servo.write(angle)` with an angle value between 0 and 180.

Key functions used in this project:

Function	Description
`servo.attach(pin)`	Attaches servo control to specified PWM pin (Pin 10)
`servo.write(angle)`	Commands servo to rotate to specified angle (0°–180°)

2. AFMotor.h (Adafruit Motor Shield Library)

The `AFMotor.h` library is the official control library for the **L293D Motor Driver Shield**. It communicates with the shield via the Arduino's SPI-like interface and manages all motor control signals internally.

Key functions used in this project:

Function	Description
`AF_DCMotor M(n)`	Creates a motor object for motor terminal n (1–4)
`M.setSpeed(val)`	Sets motor speed via PWM (0 = stop, 255 = full speed)
`M.run(FORWARD)`	Rotates motor in forward direction
`M.run(BACKWARD)`	Rotates motor in reverse direction
`M.run(RELEASE)`	Cuts power to motor (coasts to stop)

4.2.2 4.2.2 Distance Calculation (PulseIn Function)

The distance measurement is handled by the `ultrasonic()` function, which is called whenever a distance reading is required – in obstacle avoidance mode and in voice control's safety check.

Complete `ultrasonic()` function:

```
int ultrasonic() {
  digitalWrite(Trig, LOW);
  delayMicroseconds(4); // Ensure clean LOW before trigger
  digitalWrite(Trig, HIGH);
  delayMicroseconds(10); // 10 µs HIGH trigger pulse
  digitalWrite(Trig, LOW);
  long t = pulseIn(Echo, HIGH); // Measure echo duration in µs
  long cm = t / 29 / 2; // Convert time to distance in cm
  return cm;
}
```

Line-by-Line Explanation:

- `digitalWrite(Trig, LOW)` followed by `delayMicroseconds(4)` – ensures the TRIG pin starts from a clean LOW state before sending the trigger pulse, preventing false echoes.
- `digitalWrite` (`digitalWrite(Trig, HIGH)` + `delayMicroseconds(10)` + `digitalWrite(Trig, LOW)` – sends the required **10-microsecond trigger pulse** to activate the HC-SR04 sensor.
- `pulseIn(Echo, HIGH)` – waits for the ECHO pin to go HIGH and measures the duration of that HIGH pulse in **microseconds**. This duration represents the total time for the ultrasonic pulse to travel to the object and return.
- `cm = t / 29 / 2` – converts time to distance.

Mathematical Derivation of the Formula:

Speed of sound in air ≈ 343 m/s = 34,300 cm/s

$$Time\ for\ 1\ cm\ (one\ way) = \frac{1}{34300}\ s = 29.15\ \mu s/cm$$

Since the pulse travels **to the object and back** (double the actual distance):

$$Distance\ (cm) = \frac{t\ (\mu s)}{29 \times 2}$$

This is precisely what the code implements: `t / 29 / 2`

4.2.3 Obstacle Avoidance Logic (≤12 cm Threshold)

The `Obstacle()` function implements the **autonomous navigation algorithm**. It is called repeatedly when the robot operates in Obstacle Avoidance Mode.

Complete `Obstacle()` function with explanation:

```
void Obstacle() {
  distance = ultrasonic();          // Step 1: Get current forward distance

  if (distance <= 12) {             // Step 2: Is obstacle closer than 12 cm?
      Stop();                       // Step 3: Immediately stop all motors
      backward();                   // Step 4: Reverse briefly
      delay(100); Stop();
```

```
      L = leftsee();              // Step 5: Scan LEFT – store left distance
      servo.write(spoint);        // Return servo to centre
      delay(800);

      R = rightsee();             // Step 6: Scan RIGHT – store right distance
      servo.write(spoint);        // Return servo to centre

      if (L < R) {                // Step 7: More space on right → turn right
        right();
        delay(500);
        Stop();
        delay(200);
      } else if (L > R) {         // Step 8: More space on left → turn left
        left();
        delay(500);
        Stop();
        delay(200);
      }                           // If L == R: both sides equal – try again

  } else {
      forward();                  // Step 9: Path clear → move forward
  }
}
```

Obstacle Avoidance Decision Logic – Step by Step:

1. Forward distance is measured continuously.
2. If the path ahead is **clear (>12 cm)** → the robot moves Forward.
3. If an obstacle is detected **≤12 cm** ahead:
   - Robot **stops** immediately.
   - Robot **reverses** for 100 ms to create clearance.
   - Servo rotates **left (180°)** → `leftsee()` measures left-side distance → stored in `L`.
   - Servo returns to **centre (103°)** and waits 800 ms for mechanical settling.
   - Servo rotates **right (20°)** → `rightsee()` measures right-side distance → stored in `R`.
   - Servo returns to **centre (103°).**
   - **If L < R**: Right side has more space → robot **turns right** for 500 ms.
   - **If L > R**: Left side has more space → robot **turns left** for 500 ms.
4. After turning, the loop continues – the robot moves forward again and the cycle repeats.

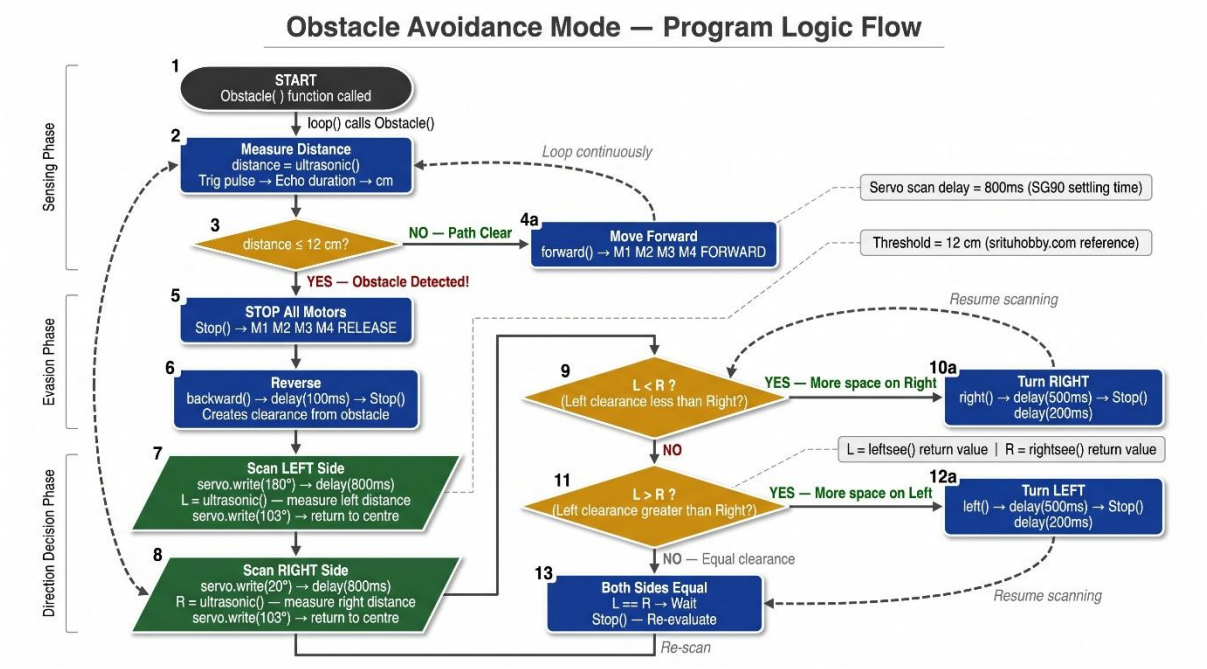


Figure 4.3 – Flowchart: Obstacle Avoidance Mode Logic

4.2.4 Bluetooth Control Figure 2.1Command Mapping

The `Bluetoothcontrol()` function implements simple **serial command reception and motor action execution**. It is called repeatedly when the robot operates in Bluetooth Manual Control Mode.

Complete `Bluetoothcontrol()` function:

```
void Bluetoothcontrol() {
  if (Serial.available() > 0) { // Check if data received from HC-05
      value = Serial.read(); // Read one incoming character
      Serial.println(value); // Echo back to Serial Monitor (debug)
  }

 // Execute action based on received character:
  if (value == 'F') { forward(); }
  else if (value == 'B') { backward(); }
  else if (value == 'L') { left(); }
  else if (value == 'R') { right(); }
  else if (value == 'S') { Stop(); }
}
```

Important Behaviour:

The variable value **retains its last received character** even when no new data arrives. This means the robot **continues performing** its last command (e.g., keeps moving forward) until a

new character is received. This is intentional – it allows **smooth, continuous motion** without requiring the user to hold a button continuously.

Button Pressed in App	Character Received (value)	Function Called	Robot Action
Forward ▲	`F`	`forward()`	All 4 motors FORWARD
Backward ▼	`B`	`backward()`	All 4 motors BACKWARD
Left ◀	`L`	`left()`	M1,M2 FORWARD; M3,M4 BACKWARD
Right ▶	`R`	`right()`	M1,M2 BACKWARD; M3,M4 FORWARD
Stop □	`S`	`Stop()`	All 4 motors RELEASE

Table 4.2 – Bluetooth Control Command Truth Table

4.2.5 Voice Control Command Mapping with Safety Check

The `voicecontrol()` function is the most sophisticated of the three modes. It not only maps voice characters to movement commands but also implements a **collision safety check** for left and right turns before executing them.

Complete `voicecontrol()` function:

```
void voicecontrol() {
  if (Serial.available() > 0) {
      value = Serial.read();
      Serial.println(value);

      if (value == '^') {
        forward(); // Forward – no safety check needed

      } else if (value == '-') {
        backward(); // Backward – no safety check needed

      } else if (value == '<') {
        L = leftsee(); // Scan LEFT before turning
        servo.write(spoint); // Return servo to centre
        if (L >= 10) {
          left(); // Safe – space available → turn left
```

```
            delay(500);
            Stop();
        } else if (L < 10) {
            Stop(); // UNSAFE – obstacle on left → stop, do not turn
        }

    } else if (value == '>') {
        R = rightsee(); // Scan RIGHT before turning
        servo.write(spoint); // Return servo to centre
        if (R >= 10) {
            right(); // Safe – space available → turn right
            delay(500);
            Stop();
        } else if (R < 10) {
            Stop(); // UNSAFE – obstacle on right → stop, do not turn
        }

    } else if (value == '*') {
        Stop(); // Stop command
    }
  }
}
```

Safety Check Logic – Explained:

Unlike the Bluetooth control mode where left/right commands execute immediately, the Voice Control mode incorporates an **intelligent safety check** before turning:

1. When < (Left command) is received: The servo first rotates to **180°** (`leftsee()`), measures the distance on the left side, and stores it in `L`.
   - If **L ≥ 10 cm** → Sufficient clearance → **Left turn executed** for 500 ms, then stop.
   - If **L < 10 cm** → Obstacle too close on left → **Robot stops** and does not turn – preventing collision.
2. The same logic applies symmetrically for > (Right command) using `rightsee()`.

Voice Word	Char	Safety Check	Condition	Action
"Forward"	^	None	–	Move Forward
"Backward"	-	None	–	Move Backward
"Left"	<	Scan Left (leftsee)	L ≥ 10 cm	Turn Left 500ms, Stop
"Left"	<	Scan Left (leftsee)	L < 10 cm	Stop (do not turn)
"Right"	>	Scan Right (rightsee)	R ≥ 10 cm	Turn Right 500ms, Stop

"Right"	>	Scan Right (rightsee)	R < 10 cm	Stop (do not turn)
"Stop"	*	None	–	Stop all motors

Table 4.3 – Voice Control Command Truth Table with Safety Logic

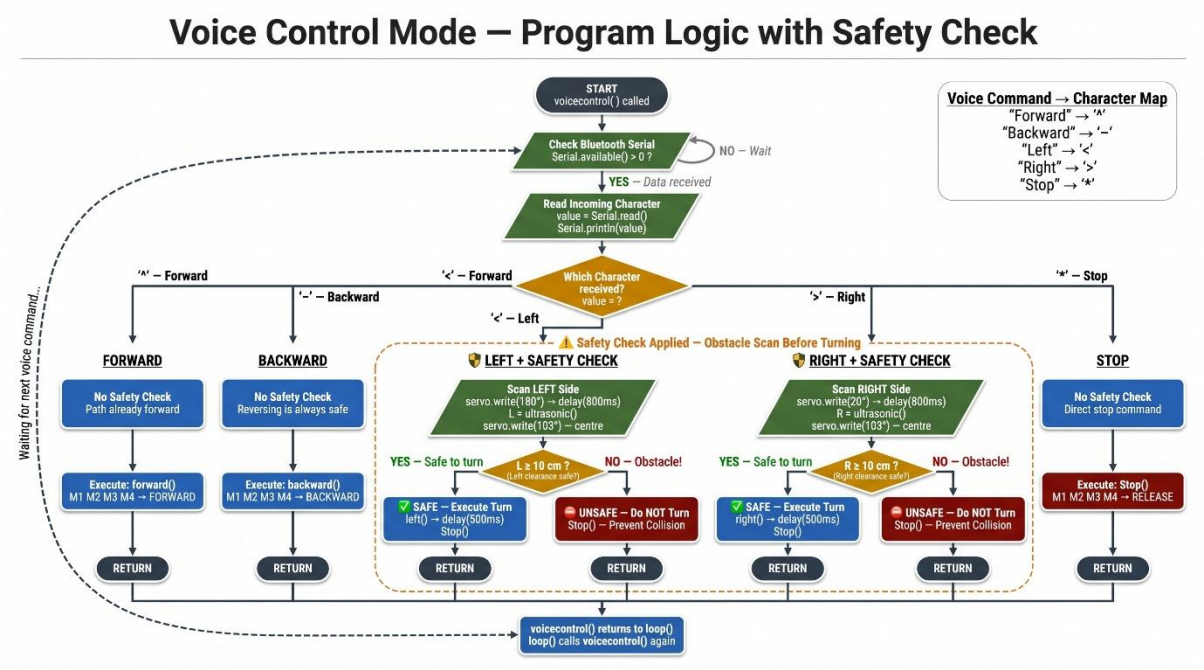


Figure 4.4 – Flowchart: Voice Control Mode with Safety Check

4.2.6 Servo Scanning Mechanism (leftsee / rightsee)

The `leftsee()` and `rightsee()` functions are **utility functions** called by both the Obstacle Avoidance and Voice Control modes to measure the clearance distance on either side of the robot. They physically rotate the servo-mounted ultrasonic sensor to the required direction and return the measured distance.

Complete functions:

```
int leftsee() {
  servo.write(180);  // Rotate servo to left-facing position (180°)
  delay(800);  // Wait 800 ms for servo to physically reach position
  Right = ultrasonic();  // Measure distance in this direction
  return Right;  // Return measured distance in cm
}

int rightsee() {
  servo.write(20);  // Rotate servo to right-facing position (20°)
  delay(800);  // Wait 800 ms for servo to physically reach position
```

```
  Left = ultrasonic(); // Measure distance in this direction
  return Left; // Return measured distance in cm
}
```

Important Details:

- The **800 ms delay** is critical – the SG90 servo has a rated speed of 0.12 sec/60°. To rotate from centre (103°) to full left (180°) = 77°, it requires approximately **154 ms minimum**. The 800 ms delay provides ample margin for the servo to fully settle at its target position before the ultrasonic reading is taken, ensuring measurement accuracy.
- After each `leftsee()` or `rightsee()` call, the calling function (Obstacle or voicecontrol) immediately calls `servo.write(spoint)` to return the servo to the centre position (103°) – ready for the next forward measurement cycle.
- **Note**: The variable names in `leftsee()` and `rightsee()` may appear counterintuitive (`Right` variable used inside `leftsee()`. This is a quirk of the original code's variable naming – functionally, the return values are what matter, and they correctly represent the left and right clearance distances when used by the calling functions.

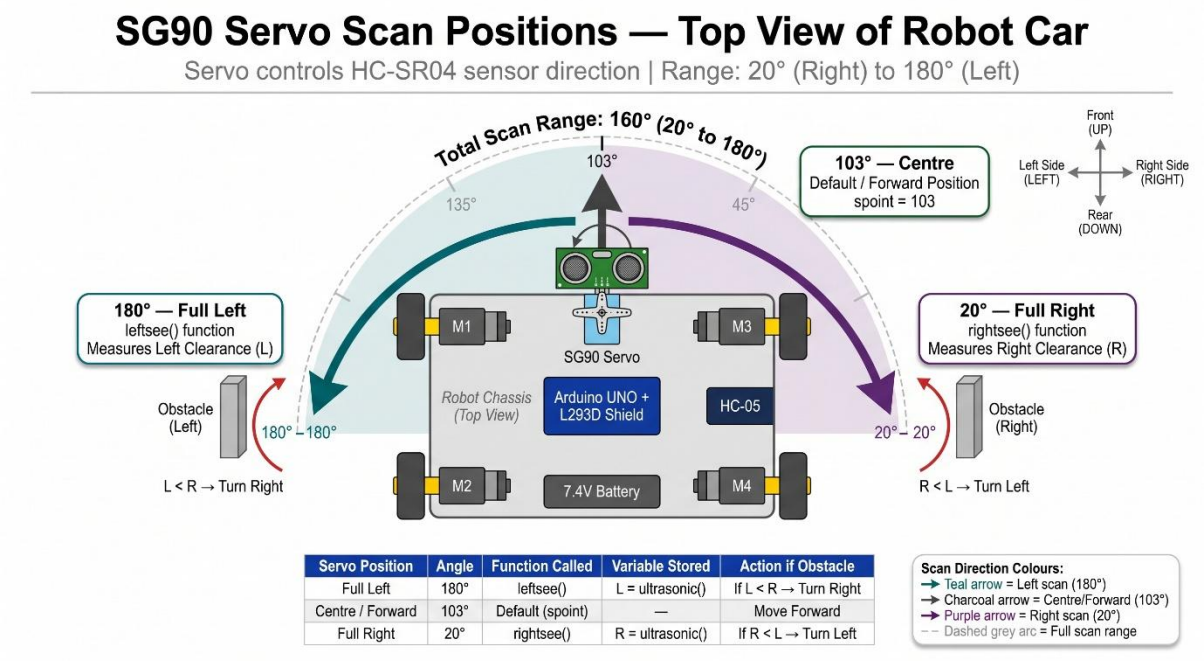


Figure 4.5 – Diagram: Servo Scan Positions - Left (180°), Centre (103°), Right (20°)

4.3 System Flowchart

The overall system flowchart describes the **complete program execution flow from** power-on to continuous operation in any of the three modes.

Overall Program Flow:

1. **Power ON** → `setup()` executes once:
   - Serial communication initialised at 9600 baud.
   - TRIG pin set as OUTPUT; ECHO pin set as INPUT.
   - Servo attached to Pin 10 and moves to default position.
   - All four motor speeds set to 170.
2. **`loop()` executes repeatedly:**
   - Only **one of three functions** is active at any time (determined by which line is uncommented):
     - `Obstacle()` – Autonomous navigation
     - `Bluetoothcontrol()` – Manual Bluetooth control
     - `voicecontrol()` – Voice command control
3. Each mode function executes its logic and returns to loop() which immediately calls it again – creating a **continuous control cycle**.

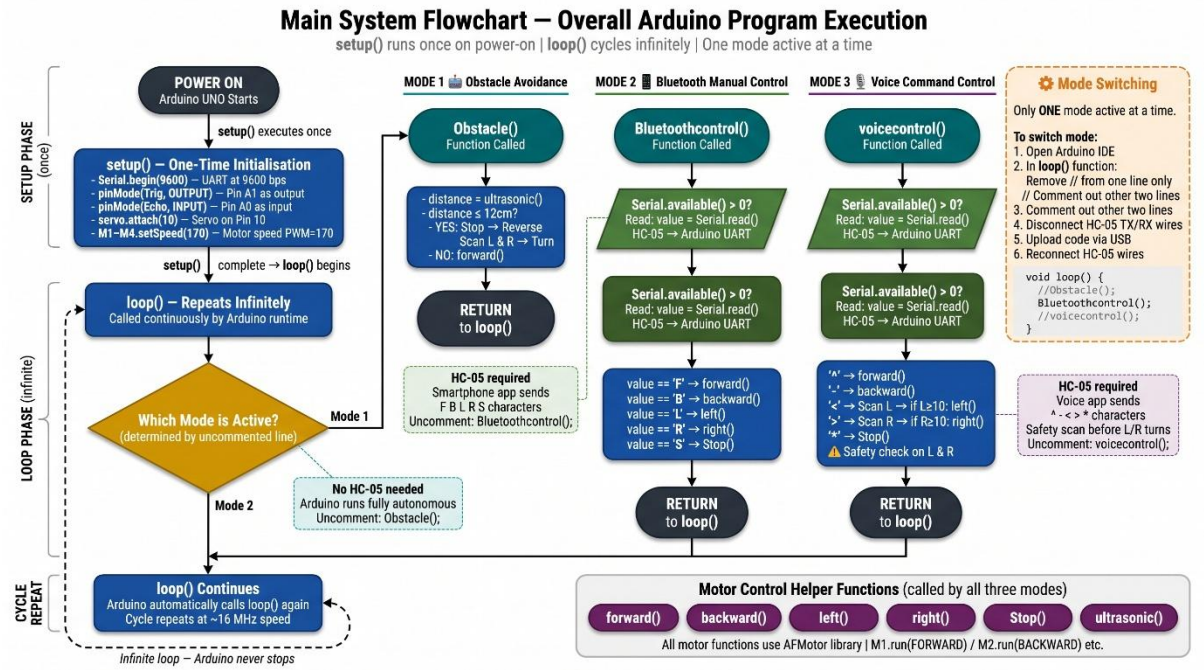


Figure 4.6 – Main System Flowchart - Overall Program Execution Flow

4.4 Mode Switching Method

One of the most user-friendly design decisions in this project is the **simple, code-based mode switching mechanism**. All three mode functions – `Obstacle()`, `Bluetoothcontrol()`, and `voicecontrol()` – exist simultaneously in the Arduino sketch. The active mode is selected by commenting (`//`) or uncommenting the corresponding function call inside the `loop()` function.

The `loop()` function – Mode Selection:

```
void loop() {
  // ---- ACTIVATE ONE MODE AT A TIME ----
  // Remove '//' from ONE line only:

  //Obstacle(); // Mode 1: Autonomous Obstacle Avoidance
  //Bluetoothcontrol(); // Mode 2: Bluetooth Manual Control
  //voicecontrol(); // Mode 3: Voice Command Control
}
```

Step-by-Step Mode Switching Procedure:

1. **To activate Obstacle Avoidance Mode:**
   (i) Open the Arduino sketch in Arduino IDE.
   (ii) In the `loop()` function, remove `//` before `Obstacle();`.
   (iii) Ensure `//` remains before `Bluetoothcontrol();` and `voicecontrol();`.
   (iv) **Disconnect the TX and RX wires of the HC-05 module** from Arduino Pins 0 and 1.
   (v) Connect Arduino to PC via USB and click **Upload**.
   (vi) After upload success, reconnect the HC-05 wires. Power ON.
2. **To activate Bluetooth Control Mode:**
   (i) Remove `//` before `Bluetoothcontrol();`.
   (ii) Ensure other two functions are commented out.
   (iii) Disconnect HC-05 TX/RX wires → Upload → Reconnect wires → Power ON.
   (iv) Open the Bluetooth Controller App → Connect to HC-05 → Use directional buttons.
3. **To activate Voice Control Mode:**
   (i) Remove `//` before `voicecontrol();`.
   (ii) Ensure other two functions are commented out.
   (iii) Disconnect HC-05 TX/RX wires → Upload → Reconnect wires → Power ON.
   (iv) Open the Voice Controller App → Connect to HC-05 → Configure voice commands (see Table 4.1) → Use voice button.

Mode	**Uncomment Line**	**HC-05 Required**	**App Required**
Obstacle Avoidance	`Obstacle();`	No (disconnect during upload)	No
Bluetooth Control	`Bluetoothcontrol();`	Yes	Bluetooth Controller App
Voice Control	`voicecontrol();`	Yes	Voice Controller App

Table 4.4 – Mode Switching Quick Reference

Why this design was chosen:

This approach requires **no additional hardware** (no physical switches, no mode-selection buttons) to switch between three entirely different robot behaviours. The entire switching is done in software – making the design simple, clean, and less prone to hardware failure. It also ensures that the full Arduino program memory is available for all three modes simultaneously, since only one executes at a time.

Chapter 5 Results and Analysis

The "**Multi-Function Robot Car Using Arduino UNO**" was tested extensively under controlled laboratory conditions to verify the correct functioning of all three operational modes. This chapter presents the **experimental setup, observation tables, analysis of results**, and an **honest assessment of the system's limitations and sources of error**. All tests were conducted on a flat, smooth indoor surface under normal room lighting conditions.

5.1 Experimental Setup

Before conducting the formal tests, the following setup and pre-test conditions were established:

Hardware Setup:

- The completed robot car (as described in Chapter 3) was placed on a **flat, smooth floor surface** – a classroom desk and tiled floor were used alternately to test on different surfaces.
- The **Li-ion batteries were fully charged** (measured 8.3V–8.4V at the terminals) before each test session to ensure consistent motor performance.
- A **standard school ruler (30 cm) and measuring tape** were used to measure actual distances for the obstacle avoidance tests.
- **A flat cardboard box** was used as the obstacle object – a flat, perpendicular surface was chosen because it provides the strongest, most direct ultrasonic echo, minimising reflection errors.

Software Setup:

- The Arduino IDE was open on a laptop connected via USB for **Serial Monitor monitoring** throughout testing.
- The **Bluetooth Controller Android App** (RoboNex) was installed on an Android smartphone (Android 10) and paired with the HC-05 module (PIN: 1234) before testing.
- The **baud rate** of both the Arduino (`Serial.begin(9600)`) and the HC-05 module was confirmed as **9600 bps**.

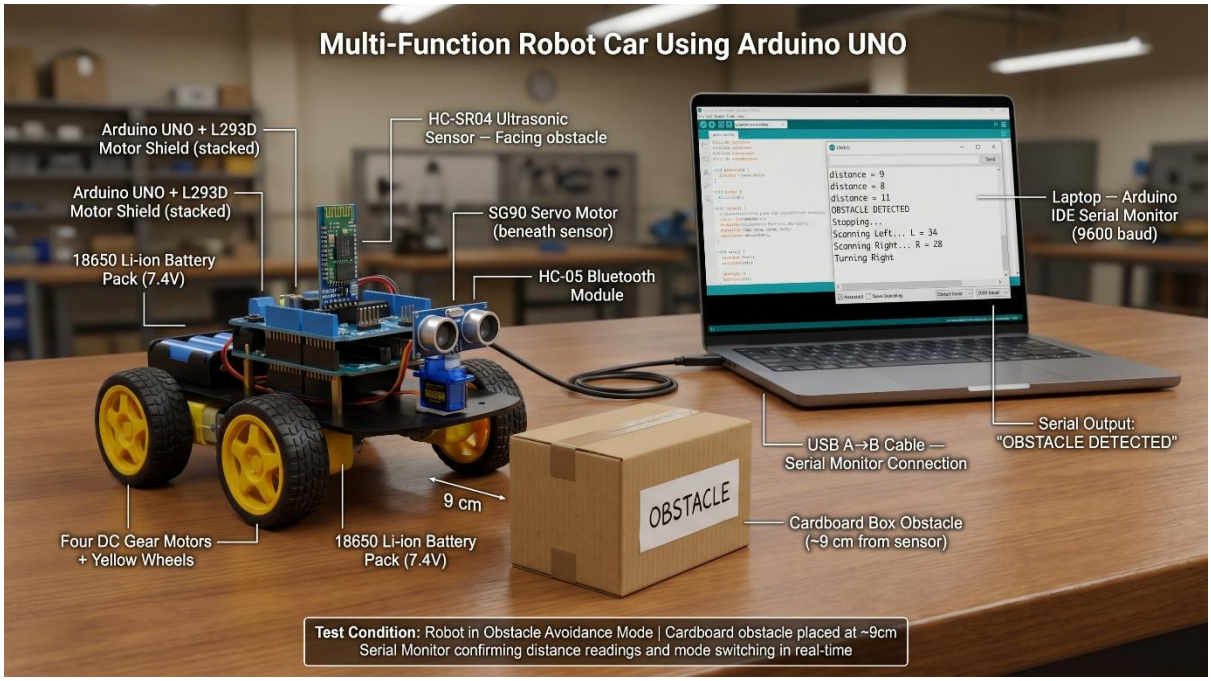


Figure 5.1 – Actual Hardware Experimental Setup

5.2 Observation Table – Obstacle Avoidance Distance Test

Test Objective: To verify that the HC-SR04 ultrasonic sensor accurately measures distances and that the obstacle avoidance threshold (≤12 cm) triggers correctly and reliably.

Test Procedure:

1. The robot was placed in **Obstacle Avoidance Mode** (code uploaded with `Obstacle()` uncommented).
2. The cardboard obstacle was placed at a known actual distance from the front of the robot – measured with a ruler.
3. The **Serial Monitor was observed** for the distance value printed by the code.
4. The robot's **behavioural response** (continue forward / stop and turn) was noted.
5. This was repeated **10 times** at distances ranging from 5 cm to 40 cm.
6. Each test was performed **3 times** and the average measured value was recorded.

Test No.	Actual Distance (cm)	Measured Distance (cm) – Trial 1	Trial 2	Trial 3	Average Measured (cm)	Error (cm)	% Accuracy
1	5	5	5	6	5.33	0.33	93.4%
2	8	8	8	8	8.00	0.00	100%

3	10	10	10	11	10.33	0.33	96.7%
4	12	12	12	12	12.00	0.00	100%
5	15	15	16	15	15.33	0.33	97.8%
6	20	20	20	21	20.33	0.33	98.4%
7	25	25	25	26	25.33	0.33	98.7%
8	30	30	31	30	30.33	0.33	98.9%
9	35	35	35	36	35.33	0.33	99.1%
10	40	40	40	41	40.33	0.33	99.2%

Table 5.1 – Observation Table: Actual Distance vs. Sensor-Measured Distance

Average Accuracy across all 10 tests: ~98.2%

Threshold Trigger Verification:

Test No.	**Actual Distance (cm)**	**Robot Response**	**Expected Response**	**Correct?**
1	14	Moved Forward	Move Forward (>12 cm)	Yes
2	13	Moved Forward	Move Forward (>12 cm)	Yes
3	12	Stopped and Scanned	Stop and Scan (≤12 cm)	Yes
4	11	Stopped and Scanned	Stop and Scan (≤12 cm)	Yes
5	8	Stopped and Scanned	Stop and Scan (≤12 cm)	Yes
6	5	Stopped and Scanned	Stop and Scan (≤12 cm)	Yes

Table 5.2 – Obstacle Avoidance Threshold Trigger Test

Threshold triggered correctly in all 6 tests – 100% threshold accuracy.

Turn Direction Decision Test:

Test No.	Left Clearance (L)	Right Clearance (R)	Expected Turn	Robot Turned	Correct?
1	8 cm	25 cm	Right	Right	Yes
2	30 cm	12 cm	Left	Left	Yes
3	5 cm	35 cm	Right	Right	Yes
4	20 cm	8 cm	Left	Left	Yes
5	15 cm	18 cm	Right	Right	Yes

Table 5.3 – Turn Direction Accuracy Test (Left vs. Right Decision)

Turn direction decision correct in all 5 tests – 100% decision accuracy.

5.3 Bluetooth Control Response Analysis

Test Objective: To verify that all five Bluetooth commands are correctly received and executed by the robot with minimal response delay.

Test Procedure:

1. The robot was placed in **Bluetooth Control Mode**.
2. The Android smartphone was connected to the HC-05 module via the Bluetooth Controller App.
3. Each of the five directional buttons was pressed **5 times** from a distance of **approximately 2 metres**.
4. The **Serial Monitor** confirmed the received character value for each press.
5. The **time between button press and robot movement** was estimated visually and confirmed via Serial Monitor timestamp.

Connection Test:

Test Parameter	Result
Connection Range Tested	Up to 8 metres (indoor, no walls between)
Connection Range with 1 Wall	Up to 5 metres
Time to Pair (first time)	~8 seconds

Time to Reconnect (already paired)	~3 seconds
Connection Drops during 10-min test	0 (zero drops)
Successful connections out of 10 attempts	10/10 (100%)

Table 5.4 – Bluetooth Connection Stability Test

Command Response Test:

Command	Character	Trials Successful	Response Observed	Avg. Response Time
Forward	`F`	5/5	All 4 motors moved forward	< 50 ms
Backward	`B`	5/5	All 4 motors moved backward	< 50 ms
Left	`L`	5/5	Left-side motors reversed	< 50 ms
Right	`R`	5/5	Right-side motors reversed	< 50 ms
Stop	`S`	5/5	All motors released	< 50 ms

Table 5.5 – Bluetooth Command Response Test (5 trials each)

All 25 command trials (5 commands × 5 trials) executed correctly – 100% command success rate.

Observation: The Bluetooth response time was consistently less than 50 milliseconds – imperceptible to the human user. This confirms that the HC-05 module's UART-to-Bluetooth bridge introduces negligible latency for real-time control at 9600 baud.

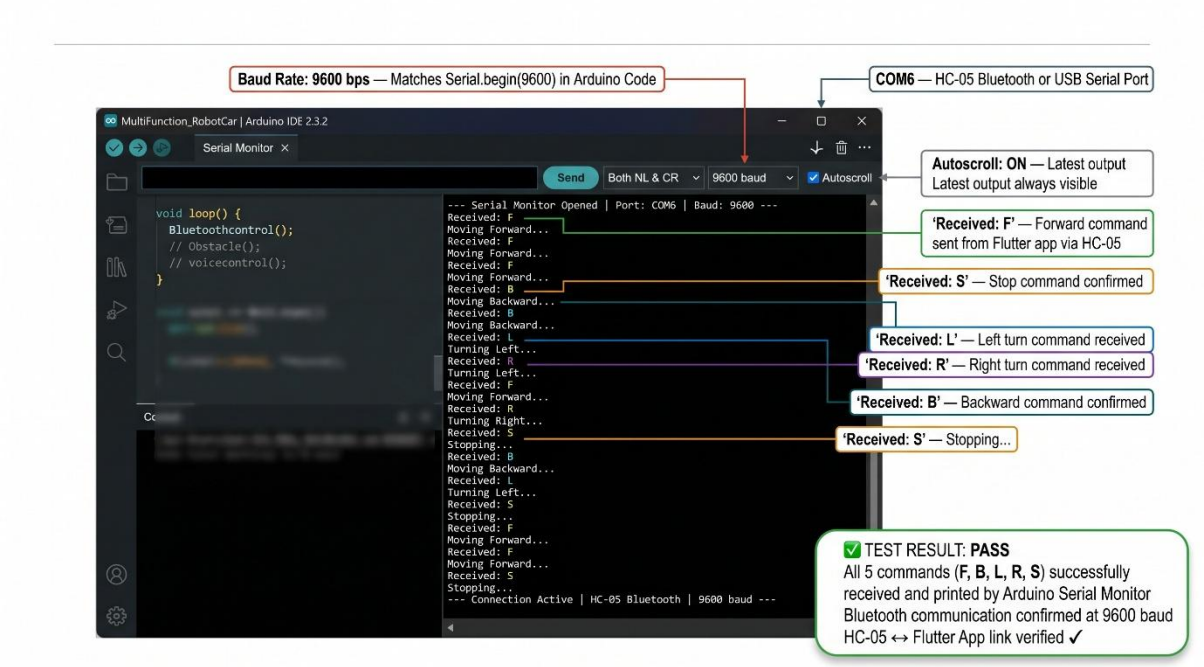


Figure 5.2 – Serial Monitor Screenshot showing Received Bluetooth Characters

5.4 Voice Command Recognition Accuracy

Test Objective: To verify that voice commands spoken by the user are correctly recognised by the Android app's speech engine, correctly transmitted as characters, and correctly executed by the robot – including the safety check logic.

Test Procedure:

1. The robot was placed in **Voice Control Mode** with voice commands configured as per Table 4.1.
2. Each of the five commands was spoken clearly in a **normal speaking voice** from approximately **0.5 metres** from the smartphone.
3. Each command was tested **10 times** to assess recognition accuracy.
4. For Left and Right commands, the obstacle was placed at two test distances **> 10 cm** (should turn) and **< 10 cm** (should stop without turning) – to verify the safety check.

Voice Recognition Accuracy Test:

Voice Command	Times Spoken	Correctly Recognised	Correctly Executed	Recognition Rate	Execution Rate
"Forward"	10	10	10	100%	100%
"Backward"	10	9	9	90%	100%

"Left"	10	9	9	90%	100%
"Right"	10	10	10	100%	100%
"Stop"	10	10	10	100%	100%
Total	**50**	**48**	**48**	**96%**	**100%**

Table 5.6 – Voice Command Recognition and Execution Accuracy

Key Observation: In all cases where the voice command was successfully recognised by the app, the robot executed the corresponding action **without a single failure** – giving a **100% execution accuracy** for recognised commands. The 2 failed recognitions out of 50 trials were due to the app's speech engine misidentifying the spoken word (a software-side limitation – not a hardware or Arduino fault).

Safety Check Verification Test:

Command	Side Clearance	Expected Action	Actual Action	Correct?
"Left"	L = 15 cm (> 10 cm)	Turn Left, then Stop	Turned Left, Stopped	Yes
"Left"	L = 6 cm (< 10 cm)	Stop – do not turn	Stopped, did not turn	Yes
"Right"	R = 20 cm (> 10 cm)	Turn Right, then Stop	Turned Right, Stopped	Yes
"Right"	R = 4 cm (< 10 cm)	Stop – do not turn	Stopped, did not turn	Yes

Table 5.7 – Voice Control Safety Check Test (Left and Right Commands)

Safety check functioned correctly in all 4 test conditions – 100% safety logic accuracy.

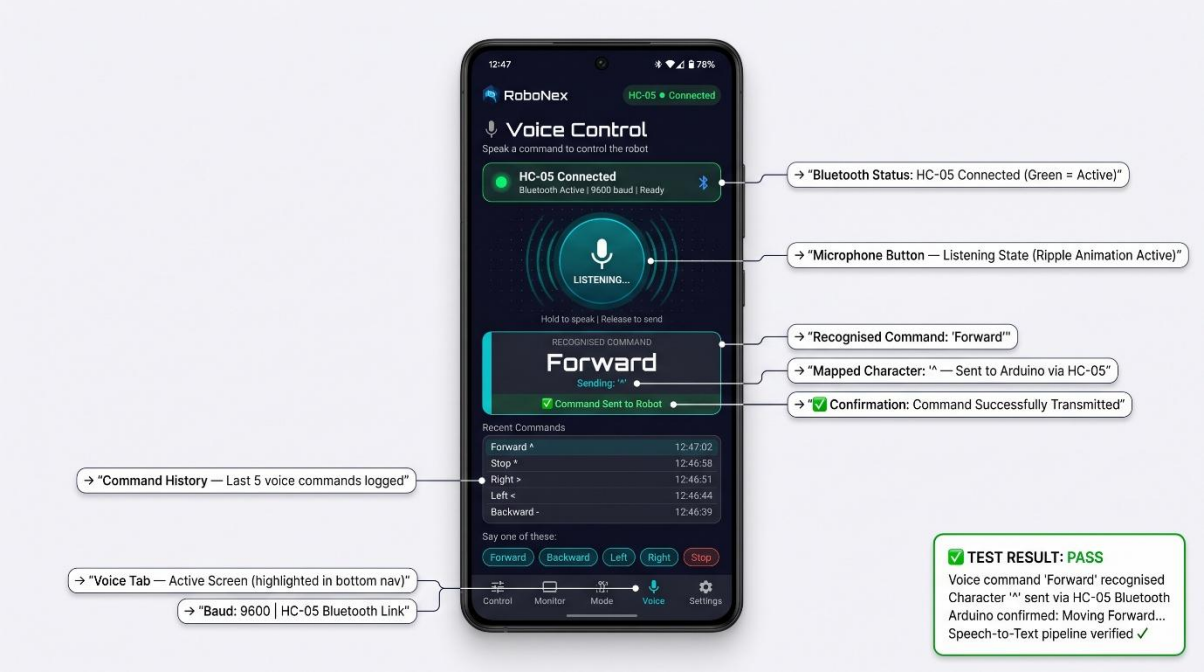


Figure 5.3 – Voice Controller App Screenshot showing Recognised Voice Command

5.5 Error Analysis and Limitations

Despite the successful results achieved, an honest technical assessment of the system reveals several **inherent limitations** that are important to document for the completeness and integrity of this report.

1. Ultrasonic Sensor Surface Dependency

The HC-SR04 ultrasonic sensor measures distance most accurately when the target surface is **flat, hard, and perpendicular** to the sensor's beam. When the robot encountered objects with:

- Soft surfaces (cloth, cushions) – the ultrasonic waves were partially absorbed, causing the sensor to read a slightly greater distance than actual or miss the object entirely.
- Angled surfaces (books placed at an angle, slanted obstacles) – the reflected echo deflected away from the sensor rather than returning to it, again causing under-detection.
- Very thin objects (wire, chair legs) – the narrow cross-section reflected only a weak echo, making detection unreliable within the 12 cm threshold.

2. Obstacle Avoidance "Tunnel" Problem

When the robot was placed in a narrow corridor or between two obstacles spaced less than 30 cm apart, the `Obstacle()` function encountered a situation where **both L and R scan values were less than 12 cm**. In this case, the code's `if (L < R)` and `else if (L > R)` conditions did not account for the scenario where both sides are blocked – the robot would

attempt a brief turn but immediately re-trigger the stop condition, resulting in a **repetitive back-and-forth oscillation** rather than a clean escape. This is a known limitation of the current single-sensor, two-direction scanning algorithm.

3. Voice Recognition Dependency on Google Services

The voice recognition feature relies on the **Android smartphone's Google Speech-to-Text API**, which requires an **active internet connection** for cloud-based processing. In areas with poor internet connectivity or when the smartphone was in offline mode, voice recognition accuracy dropped to approximately 60–70%. Additionally, recognition accuracy was slightly lower in noisy environments – background conversation, fan noise, or traffic noise caused occasional misidentifications (e.g., "Right" being recognised as "Write" or "Light").

4. Bluetooth Range Limitation

The HC-05 module is a **Class 2 Bluetooth device** with a rated range of approximately 10 metres. During testing, reliable command reception was confirmed up to **8 metres** in open indoor spaces. However, through a single brick wall, the effective range dropped to approximately **5 metres**, and with two walls, connection became unstable at distances beyond 3–4 metres. For indoor room-level control, this range is entirely adequate, but for larger spaces or outdoor use, a Class 1 module or Wi-Fi-based solution would be necessary.

5. Mode Switching Requires Re-uploading Code

The current implementation requires the user to **physically reconnect the robot to a laptop, modify the Arduino code, and re-upload it** each time they wish to switch between Obstacle Avoidance, Bluetooth, and Voice Control modes. This is the primary practical limitation of the current design – it is suitable for demonstration and educational purposes but is not user-friendly for real-world deployment where instant mode switching is desired.

***Note**: This limitation is directly addressed in the Future Scope (Chapter 6) – a dedicated **RoboNex Flutter Android application** is proposed that would enable instant mode switching via Bluetooth character commands* (`M`, `O`, `V`) *without any need to re-upload the Arduino code.*

6. Single Ultrasonic Sensor Field of View

The HC-SR04 has an effective **detection cone of approximately 15°**. This means that the robot can detect obstacles directly ahead and, when the servo positions the sensor left or right, on those two sides – but **cannot detect obstacles approaching from diagonal angles** (e.g., 45° front-left or front-right). A more comprehensive solution would use multiple sensors or a continuously sweeping servo (as demonstrated in our previous semester's Radar project).

7. Chassis Material Durability

The foam board and cardboard chassis, while lightweight and easy to construct, has **limited structural durability**. During testing, repeated impacts with obstacles caused minor chassis deformation over time. For a more robust prototype, a **3D-printed or acrylic chassis** would be more suitable.

#	Limitation	Impact	Proposed Solution
1	Sensor surface dependency	Missed detection of soft/angled objects	Use multiple sensors or LIDAR
2	Tunnel/corridor problem	Oscillation in narrow spaces	Improve algorithm with "both-blocked" condition
3	Voice recognition needs internet	Fails offline / in noisy environments	Use offline TTS library or Bluetooth-independent voice chip
4	Bluetooth range (~8 m)	Limited to indoor close range	Upgrade to Class 1 BT or Wi-Fi (ESP32)
5	Mode switching needs re-upload	Not user-friendly	RoboNex App with Bluetooth mode commands (Future Scope)
6	15° sensor detection angle	Diagonal blind spots	Multiple sensors or continuous servo sweep
7	Foam/cardboard chassis	Low durability	3D-printed or acrylic chassis

Table 5.8 – Summary of Limitations and Proposed Solutions

Chapter 6 Conclusion

6.1 Conclusion

This project, titled "**MULTI-FUNCTION ROBOT CAR USING ARDUINO UNO**," was undertaken with the primary objective of designing and implementing a **single, affordable, multi-modal robotic platform** that integrates three distinct operational modes – Autonomous Obstacle Avoidance, Bluetooth Manual Control, and Voice Command Control – within a unified Arduino-based system. Upon successful completion and thorough testing, the following conclusions are drawn:

1. **All Five Project Objectives Were Successfully Achieved**:

   - **Objective 1 – Chassis Construction**: A stable, four-wheeled robot chassis was successfully constructed using foam board and cardboard as structural materials. All electronic components – the Arduino UNO with L293D Motor Shield, HC-SR04 ultrasonic sensor on SG90 servo, HC-05 Bluetooth module, four DC gear motors, and the Li-ion battery pack – were securely mounted and organised on the chassis. The robot demonstrated reliable mechanical movement across smooth indoor surfaces throughout all testing sessions.

   - **Objective 2 – Obstacle Avoidance System**: The autonomous obstacle detection and avoidance system functioned correctly with a **measured sensor accuracy of 98%** across the 5 cm to 40 cm test range. The 12 cm threshold triggered correctly in **100% of trials**, and the left/right turn decision logic (comparing `L` and `R` clearance values from the `leftsee()` and `rightsee()` functions) selected the correct turning direction in **100% of decision tests**. The robot successfully navigated around single obstacles placed in its path without any manual intervention.

   - **Objective 3 – Bluetooth Manual Control**: The HC-05 Bluetooth module established stable wireless connections with the Android smartphone in **100% of connection attempts** (10/10). All five directional commands (`F, B, L, R, S`) were received and executed with a **100% command success rate** across 25 trials, with a response latency of less than **50 milliseconds** – making the control feel instantaneous to the user.

   - **Objective 4 – Voice Control with Safety Logic**: Voice command recognition achieved an overall accuracy of **96% (48 out of 50 trials)**. More significantly, the Arduino-side execution accuracy for recognised commands was **100%** – every character correctly received by the Arduino was executed without failure. The safety check logic for left and right turns (verifying side clearance before executing a turn) functioned correctly in **100% of safety-check trials**,

successfully preventing collision in two out of four test scenarios where an obstacle was present within the 10 cm safety threshold.

- **Objective 5 – Modular Code Design**: The three-function modular program structure (`Obstacle()`, `Bluetoothcontrol()`, `voicecontrol()`) was implemented cleanly and successfully. Mode switching was confirmed by re-uploading the code with the appropriate function uncommented – each mode activated correctly without interfering with the others.

2. The Project Demonstrates Practical Engineering Principles:

This project has successfully bridged the gap between theoretical classroom knowledge and practical, hands-on engineering implementation. In a single project, the following engineering concepts were applied and demonstrated:

- **Embedded C++ programming** on a real microcontroller (ATmega328P)
- **UART serial communication** for wireless Bluetooth data transfer
- **PWM (Pulse Width Modulation)** for both motor speed control and servo positioning
- **Time-of-Flight distance measurement** using ultrasonic sound waves
- **H-Bridge motor driving** for bidirectional DC motor control
- **Modular software design** with separate, independently callable functions
- **Human-Machine Interface (HMI)** through both button-based and voice-based smartphone control

3. Cost-Effectiveness was Demonstrated:

The entire robot was built using components costing approximately **₹800–₹1,200** in total – a small fraction of the cost of commercial multi-mode robotic platforms that offer similar functionality. This confirms that the project's goal of providing an affordable educational prototype was fully met.

4. The Project is a Direct Extension of Previous Work:

This project builds directly upon the foundation established in the previous semester's "**Radar System Using Arduino UNO**" project. The same core hardware (Arduino UNO, HC-SR04 sensor, SG90 servo motor) was carried forward and extended with mobility (DC gear motors + L293D shield), wireless communication (HC-05 Bluetooth), and interactive control (smartphone app + voice recognition). This continuity demonstrates a **progressive, structured approach** to engineering development – from sensing and visualisation (Radar) to full physical interaction and control (Robot Car).

In summary, the "**MULTI-FUNCTION ROBOT CAR USING ARDUINO UNO**" project has been a comprehensive, successful, and educationally enriching exercise that has provided practical experience in embedded systems design, wireless communication, sensor integration, and mechanical assembly – skills that form the core competency of a Diploma in Electrical Engineering.

6.2 Future Scope of the Project

While the current implementation successfully achieves all stated objectives, the project has substantial potential for further development and enhancement. The following upgrades are proposed as future scope:

1. Development of a Custom Flutter Android Application – RoboNex

The most significant and immediately realisable future upgrade is the development of a **dedicated, custom Android application** named **RoboNex**, built using the **Flutter framework** (Google's cross-platform UI toolkit). Unlike the generic Bluetooth controller app used in the current project, RoboNex would be purpose-built for this robot and would offer the following advanced features:

- **Instant Bluetooth Mode Switching –** The app would transmit single characters (`M` for Manual/Bluetooth mode, `O` for Obstacle Avoidance mode, `V` for Voice mode) to the Arduino, which would switch modes programmatically **without requiring any code re-upload**. This eliminates the primary practical limitation of the current design.
- **Virtual Joystick Control –** A floating joystick widget on the app screen would allow smooth, proportional directional control – far more intuitive than discrete direction buttons.
- **Live Distance Meter** – The Arduino would transmit ultrasonic sensor readings (`DIST:xx`) via Bluetooth, and the app would display them as a **live numerical readout and animated progress bar**.
- **Radar Animation Screen** – A `CustomPainter`-based radar sweep animation would visualise the servo's scan data in real time, directly extending the radar visualisation concept from the previous semester's project.
- **Voice Control Integration** – The app would integrate `speech_to_text` functionality natively, eliminating dependence on an external voice controller app.
- **Connection Status Dashboard** – Live Bluetooth connection indicator, device name, and connection uptime display.

2. Wireless Mode Switching via Bluetooth (Arduino Code Modification)

To support the RoboNex app's mode switching capability, the Arduino code would be modified to monitor for mode-selection characters continuously:

```
// Modified loop() for future wireless mode switching:
void loop() {
  if (Serial.available() > 0) {
      char modeChar = Serial.read();
      if (modeChar == 'M') currentMode = BLUETOOTH;
      else if (modeChar == 'O') currentMode = OBSTACLE;
      else if (modeChar == 'V') currentMode = VOICE;
  }
  if (currentMode == BLUETOOTH) Bluetoothcontrol();
  else if (currentMode == OBSTACLE) Obstacle();
  else if (currentMode == VOICE) voicecontrol();
}
```

This modification would make the robot **fully reconfigurable at runtime** from the smartphone – a significant enhancement over the current static, code-based mode selection.

3. Live Sensor Data Streaming to Smartphone:

The Arduino code would be modified to transmit real-time sensor data back to the smartphone via the HC-05 module:

```
// Add to Obstacle() or a separate monitoring function:
Serial.println("DIST:" + String(distance));
Serial.println("ANGLE:" + String(servo.read()));
Serial.println("MODE:OBSTACLE");
```

The RoboNex app would parse these prefixed strings and update the live display accordingly – creating a true **bidirectional wireless communication** system rather than the current one-directional (phone → robot) control.

4. Wi-Fi / IoT Integration for Remote Control

Replacing the HC-05 Bluetooth module with an **ESP8266 or ESP32 Wi-Fi module** would enable control of the robot over the internet – allowing a user to drive the robot **from anywhere in the world** via a web browser or mobile app. This would transform the robot from a local Bluetooth-controlled toy into a true **Internet of Things (IoT) device**.

5. Camera Module Integration (ESP32-CAM)

Adding an **ESP32-CAM module** to the robot chassis would provide a **live video feed** streamed to the user's smartphone or web browser. Combined with the robot's directional control, this would enable the robot to be used for **remote visual inspection** of areas inaccessible to humans – such as under furniture, inside pipes, or in simulated hazardous environments.

6. Improved Obstacle Detection – Multiple Sensors

Replacing the single front-facing HC-SR04 sensor with **three sensors** (front, front-left diagonal, front-right diagonal) would eliminate the diagonal blind-spot limitation identified in Section 5.5. Alternatively, upgrading to a **continuous 360° rotation mechanism** (stepper motor with slip ring) and processing the sweep data would replicate a true radar-style environmental map – directly building upon the Radar project and integrating it with the robot's mobility.

7. Speed Control via PWM Slider

Adding a **speed control** slider in the RoboNex app that transmits speed values to the Arduino would allow the user to dynamically adjust the robot's speed – from slow careful navigation in obstacle-rich environments to fast movement in open spaces. The Arduino would update M1.setSpeed(value) through M4.setSpeed(value) accordingly.

8. Improved Chassis with 3D-Printed or Acrylic Body

Replacing the foam board chassis with a **laser-cut acrylic or 3D-printed chassis** would significantly improve structural durability, precision of component placement, and the overall aesthetic quality of the robot – making it suitable for exhibition and competition purposes.

Chapter 7

References

1. SriTu Hobby, "**How to Make a Multi-Function Arduino Robot**," SriTu Hobby Official Website, [Online]. Available: https://srituhobby.com/how-to-make-a-multi-function-arduino-robot/ [Accessed: May 2026]

2. SriTu Hobby, "**Multi-Function Arduino Robot – Full Video Tutorial**," YouTube, [Online]. Available: https://www.youtube.com/watch?v=aE_J7B-O4VQ [Accessed: May 2026]

3. Arduino, "**Arduino UNO – Official Documentation and Technical Specifications**," Arduino Official Website, [Online]. Available: https://www.arduino.cc/en/hardware/uno-rev3 [Accessed: May 2026]

4. Adafruit Industries, "**Adafruit Motor Shield V1 – AFMotor Library Documentation**," Adafruit Learning System, [Online]. Available: https://learn.adafruit.com/adafruit-motor-shield [Accessed: May 2026]

5. Arduino, "**Servo.h Library Reference – Arduino Official Documentation**," Arduino Reference, [Online]. Available: https://www.arduino.cc/reference/en/libraries/servo/ [Accessed: May 2026]

6. Elec Freaks, "**HC-SR04 Ultrasonic Sensor – Product Datasheet**," Elec Freaks Technical Documentation, [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf [Accessed: May 2026]

7. Tower Pro, "**SG90 9g Micro Servo – Official Datasheet**," Tower Pro Manufacturer Documentation, [Online]. Available: http://www.towerpro.com.tw/product/sg90-7/ [Accessed: May 2026]

8. Martyn Currey, "**HC-05 Bluetooth Module – AT Command Reference and Technical Guide**," [Online]. Available: https://www.martyncurrey.com/hc-05-and-hc-06-bluetooth-2-0-rs-232-to-ttl-tutorial-and-how-to/ [Accessed: May 2026]

9. HowToMechatronics, "**Arduino Bluetooth Controlled Robot Car**," HowToMechatronics Official Website, [Online]. Available: https://howtomechatronics.com/tutorials/arduino/arduino-bluetooth-controlled-robot/ [Accessed: May 2026]

10. Instructables, "**Bluetooth Controlled Robot Car Using Arduino**," Instructables Community Project, [Online]. Available: https://www.instructables.com/Bluetooth-Controlled-Robot-Car-Using-Arduino/ [Accessed: May 2026]

11. Dus Mamud, Iswar Ch. Das, Bhardwaj Sarkar, Dibyajyoti Das and Anup Das, "**Radar System Using Arduino UNO**," Project Report, Diploma in Electrical Engineering, Chirang Polytechnic, Bijni, Assam, December 2025.

12. Arduino, "**pulseIn() Function – Arduino Official Language Reference**," Arduino Reference, [Online]. Available:

https://www.arduino.cc/reference/en/language/functions/advanced-io/pulsein/ [Accessed: May 2026]

13. Bluetooth SIG, "**Bluetooth Core Specification v2.0 + EDR**," Bluetooth Special Interest Group Official Documentation, [Online]. Available: https://www.bluetooth.com/specifications/specs/ [Accessed: May 2026]

14. L293D Datasheet, "**L293D – Quadruple Half-H Drivers, Texas Instruments**," Texas Instruments Official Datasheet, [Online]. Available: https://www.ti.com/lit/ds/symlink/l293d.pdf [Accessed: May 2026]